

IOT BASED ENERGY MONITORING AND REMOTE LOAD CONTROL FOR SUSTAINABLE INDUSTRY

A Project Report

*Submitted to the FACULTY of ENGINEERING of
JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY, KAKINADA*

in partial fulfillment of the requirements,

for the award of the Degree of

Bachelor of Technology

in

Internet of Things

By

**Seelam Santhosh
(23485A6021)**

**Yarramsetty Anusha
(22481A60B3)**

**Mailapalli Naveen
(22481A6058)**

**Peddiboyina Banu Sai Teja
(22481A6089)**

Under the Guidance of

Dr. T. Ananda Babu

Associate Professor



Department of Internet of Things

SESHADRI RAO GUDLAVALLERU ENGINEERING COLLEGE

(An Autonomous Institute with Permanent Affiliation to JNTUK, Kakinada)

SESHADRI RAO KNOWLEDGE VILLAGE

GUDLAVALLERU - 521356

ANDHRA PRADESH

2025-26

Department of Internet of Things
SESHADRI RAO GUDLAVALLERU ENGINEERING COLLEGE
(An Autonomous Institute with Permanent Affiliation to JNTUK, Kakinada)
SESHADRI RAO KNOWLEDGE VILLAGE
GUDLAVALLERU – 521356



CERTIFICATE

This is to certify that the project report entitled “**IOT BASED ENERGY MONITORING AND REMOTE LOAD CONTROL FOR SUSTAINABLE INDUSTRY**” is a bonafide record of work carried out by **S.Santhosh (23485A6021), Y.Anusha (22481A60B3), M.Naveen (22481A6058), P.Banu Sai Teja (22481A6089)** under my guidance and supervision in partial fulfillment of the requirements, for the award of the degree of Bachelor of Technology in **Internet of Things** of **Seshadri Rao Gudlavalleru Engineering** affiliated to **Jawaharlal Nehru Technological University, Kakinada**.

Dr. T. Ananda Babu
Project Guide

Dr. Y. Syamala
Head of the Department

ACKNOWLEDGEMENT

We are very glad to express our deep sense of gratitude to **Dr.T.Ananda Babu**, Associate Professor, Internet of Things for guidance and cooperation for completing this project. We convey our heartfelt thanks to him for inspiring assistance till the end of our project.

We convey our sincere and indebted thanks to our beloved Head of the Department **Dr. Y. Syamala**, for her encouragement and help for completing our project successfully.

We also extend our gratitude to our Principal **Dr. B. Karuna Kumar**, for the support and for providing facilities required for the completion of our project.

We impart our heartfelt gratitude to all the Lab Technicians for helping us in all aspects related to our project.

We thank our friends and all others who rendered their help directly and indirectly to complete our project.

S. Santhosh (23485A6021)

Y. Anusha (22481A60B3)

M. Naveen (22481A6058)

P. Bhanu Sai Teja (22481A6089)

CONTENTS

TITLE	PAGE No.
LIST OF FIGURES	iii
LIST OF TABLES	iv
NOMENCLATURE	v
ABSTRACT	vi
CHAPTER 1 - INTRODUCTON	
1.1 Background	1
1.2 Aim of this Project	2
1.3 Significance of this Work	2
1.4 Methodology	2-3
1.5 Outline of this Report	4
1.6 Conclusion	4
CHAPTER 2 - LITERATURE REVIEW	5-7
CHAPTER 3 - IOT BASED ENERGY MONITTING AND REMOTE LOAD CONTROL FOR SUSTAINABLE INDUSTRY	
3.1 Problem Statement	8
3.2 Existing System	9
3.3 Proposed System	9-10
CHAPTER 4 - HARDWARE COMPONENTS	
4.1 Arduino UNO	11-16
4.2 GSM Module	17
4.3 Energy meter	17-19
4.4 Relay	19-20
4.5 16X2 LCD DIsplay	21
4.6 ESP8266 IoT Module	22-23
4.7 AC Loads	23-24

CHAPTER 5 - HARDWARE IMPLEMENTATION	
5.1 Circuit Diagram	25
5.2 Hardware Design	26
5.3 Output Voltages	27
CHAPTER 6 - SOFTWARE IMPLEMENTATION	
6.1 Arduino IDE	28
6.2 Arduino IDE:Initial Setup	29
6.3 IDE Board Setup	29
6.4 IDE COM Port Setup	30
6.5 Testing	30
6.7 Real Time Server Setup	31
CHAPTER 7 - RESULTS AND DISCUSSION	32
CONCLUSION AND FUTURE SCOPE	33
BIBLIOGRAPHY	34
PROJECT OUTCOMES (COs) MAPPED WITH PSOs, POs and SDGs	35-38
APPENDIX A - Source Code	39-49
APPENDIX B - Paper and Certificates	50-58

LIST OF FIGURES

3.3	Flow Chart	10
4.1	Arduino UNO Board	11
4.1	Arduino IDE	13
4.2	GSM Module	17
4.3	Energy Meter	17
4.4	Relay	19
4.5	LCD Display	21
4.6	ESP8266 Module	22
4.7	AC Fan	23
4.8	AC Bulb & Bulb Holder	24
4.9	AC Socket	24
5.1	Circuit Diagram	25
5.2	Hardware Design	26
6.6	Load Monitoring in Server	31
7.1	System Output	32

LIST OF TABLES

TABLE	NAME OF THE TABLE	PAGE NO
5.3	Output Voltages	27

NOMENCLATURE

IoT – Internet of Things

MCU – Microcontroller Unit

Wi-Fi – Wireless Fidelity

MQTT – Message Queuing Telemetry Transport

HTTP – Hypertext Transfer Protocol

PF – Power Factor

AC – Alternating Current

DC – Direct Current

kWh – Kilowatt-hour

RTC – Real Time Clock

LCD – Liquid Crystal Display

GPIO – General Purpose Input Output

ABSTRACT

Efficient energy management has become a critical requirement for industries aiming to reduce operational costs and promote sustainable development. This project presents an IoT-Based Energy Monitoring and Remote Load Control System for Sustainable Industry, designed to monitor electrical energy consumption in real time and provide intelligent control over connected loads. The system integrates an Arduino UNO microcontroller with a single-phase energy meter to measure important electrical parameters such as voltage, current, power, and energy consumption the measured data is displayed locally using a 16×2 LCD and simultaneously transmitted to a cloud-based real-time server through the ESP8266 Wi-Fi module. This IoT integration enables remote monitoring, data logging, and analysis of energy consumption patterns, helping industries identify inefficiencies and optimize electricity usage. Additionally, a GSM module (SIM900) is incorporated to send SMS alerts in case of overload, abnormal energy usage, or power failure, ensuring reliable communication even in the absence of internet connectivity. Relay modules are used to control AC loads automatically or remotely, enhancing safety and preventing equipment damage during fault conditions. The proposed system is designed to be cost-effective, scalable, and easy to implement, making it suitable for industries of varying sizes as well as commercial establishments by combining real-time monitoring, IoT analytics, remote load control, and dual communication (Wi-Fi and GSM), the system provides a comprehensive solution for sustainable energy management. The project contributes to improved operational efficiency, reduced electricity costs, enhanced system reliability, and environmentally responsible energy utilization.

Keywords: *GSM (Global System for Mobile Communications), IoT (Internet of Things), Smart Energy Monitoring, Energy Management System.*

CHAPTER 1

INTRODUCTION

1.1 Background

With the rapid growth of population and industrialization, the demand for electrical energy is increasing day by day. Traditional energy meters only measure power consumption and require manual reading, which is time-consuming, inefficient, and prone to errors. There is a strong need for an intelligent system that can monitor energy usage in real time and allow remote control of electrical loads.

The IoT Based Smart Energy Meter with GSM is designed to overcome these limitations by providing an automated and remote energy monitoring solution. This project uses Arduino as the main microcontroller, along with GSM and Wi-Fi communication technologies, to measure, monitor, and control electrical energy consumption efficiently.

In this system, a single-phase energy meter is used to measure electrical energy. An LDR sensor detects the blinking pulses of the energy meter LED, which are proportional to the consumed power. These pulses are counted by the Arduino to calculate total energy usage accurately. The measured data is then transmitted to the user through GSM SMS alerts and uploaded to the cloud using an ESP8266 Wi-Fi module for real-time monitoring.

Additionally, the system provides remote control of electrical loads using GSM commands. Three AC loads are connected through relay modules, allowing the user to turn appliances ON or OFF remotely using simple SMS commands. This feature helps in reducing unnecessary power consumption and improves energy management.

Overall, this project offers a low-cost, reliable, and efficient smart energy management solution suitable for smart homes, industries, and smart grid applications.

1.2 Aim of the Project

The main aim of this project is to design and implement an IoT-based system for monitoring energy consumption and controlling electrical loads remotely in an industrial environment.

The objectives of the project are:

- 1) To measure voltage, current, power, and energy consumption
- 2) To display the measured values locally using an LCD
- 3) To transmit energy data to the cloud using IoT
- 4) To control electrical loads remotely using a relay
- 5) To reduce power wastage and improve energy efficiency

1.3 Significance of this Work

This project is significant because it provides a smart solution for managing energy in industries. By enabling real-time monitoring, the system helps identify power-hungry machines and abnormal energy usage. Remote load control improves safety and reduces the need for manual operation in high-voltage environments.

The system supports sustainable development by reducing unnecessary power consumption and lowering carbon emissions. It also helps industries save money by optimizing energy usage. This work demonstrates how IoT technology can be effectively applied to solve real-world energy management problems.

1.4 Methodology

The methodology of the proposed IoT-based smart energy meter focuses on energy measurement, data processing, communication, and remote load control. The complete system operation is divided into several stages as described below.

1. System Design and Hardware Integration

The system is designed using Arduino as the central controller. A single-phase energy meter is connected to the AC supply, and an LDR sensor is placed near the energy meter's LED to detect light pulses generated during power consumption. These pulses

are fed to the Arduino through a signal conditioning circuit. three relay modules are connected to the Arduino to control three AC loads. Proper electrical isolation is maintained between the low-voltage control circuit and high-voltage AC loads.

2. Energy Pulse Detection and Calculation:

The energy meter generates LED pulses proportional to the energy consumed. The LDR sensor detects these pulses, and the Arduino counts them using interrupt or polling methods. Based on the meter's pulse rate (pulses per unit), the Arduino calculates the total energy consumption in units (kWh).

3. Communication Using GSM Module

A GSM module is interfaced with the Arduino via serial communication. The GSM module performs two main functions:

- a) Receives SMS commands from the user to control electrical loads.
- b) Sends SMS alerts containing energy consumption data or system notifications.

The Arduino continuously monitors incoming SMS messages, decodes the commands, and executes the corresponding load control operations.

4. IoT Data Transmission Using ESP8266

The ESP8266 Wi-Fi module is used to connect the system to the internet. Energy consumption data calculated by the Arduino is transmitted to a cloud server or IoT platform at regular intervals. This allows users to monitor real-time energy usage remotely through a web or mobile application.

5. Remote Load Control

Three AC loads are controlled using relay modules. Based on SMS commands received via GSM, the Arduino switches the relays ON or OFF. This enables remote control of appliances without physical interaction, improving energy efficiency.

1.5 Outline of this Report

This report is organized into different chapters as follows:

Chapter 1: Introduction

Presents background, aim, significance, methodology, and report structure.

Chapter 2: Literature Survey

Reviews previous work related to IoT and energy monitoring systems.

Chapter 3: Hardware Components

Describes block diagram, hardware components.

Chapter 4: Hardware Implementation

Explains circuit design and system integration.

Chapter 5: Software Implementation

Describes Programming and Software Design

Chapter 6: Results and Discussion

Presents experimental results and performance analysis.

1.6 Conclusion

The IoT Based Smart Energy Meter with GSM and Wi-Fi project successfully demonstrates an intelligent and efficient solution for real-time energy monitoring and remote load control. By integrating Arduino as the main microcontroller, along with GSM and ESP8266 Wi-Fi modules, the system overcomes the limitations of conventional energy metering systems.

The proposed system accurately measures energy consumption using a single-phase energy meter and an LDR-based pulse sensing technique. The calculated energy data is transmitted to users through SMS alerts using GSM and uploaded to a cloud platform via Wi-Fi, enabling real-time monitoring from any location.

CHAPTER 2

LITERATURE REVIEW

[1] A. K. Pandey, R. Singh, and P. Sharma present an IoT-based smart energy monitoring system designed for industrial and residential environments. The study integrates smart sensors with microcontrollers to monitor voltage, current, and power consumption in real time. The collected data is transmitted through Wi-Fi modules to cloud platforms where it can be visualized using web dashboards and mobile applications. The system enables users to track electricity consumption patterns and identify energy wastage. Experimental results demonstrate improved energy management and reduced operational costs through continuous monitoring and remote access. The proposed framework highlights the benefits of IoT technologies in improving energy efficiency and sustainable energy utilization.

[2] S. Kumar and M. Gupta present a remote load control system using IoT technology for efficient power management in industrial environments. The proposed system utilizes an Arduino-based controller integrated with Wi-Fi communication to control electrical appliances remotely through a web interface. The study emphasizes automation and user convenience by allowing operators to switch loads ON or OFF from remote locations. Real-time monitoring of electrical parameters helps detect abnormal consumption patterns and prevent electrical faults. The results show that the system can significantly reduce energy wastage and improve operational efficiency in industrial facilities.

[3] J. Lee and K. Park present a smart energy monitoring framework using wireless sensor networks and IoT platforms. The research focuses on integrating distributed energy meters with cloud computing to analyze power consumption data. The system collects data from multiple sensors deployed across different locations and transmits it to a central server for analysis. The study highlights the role of data analytics in identifying peak consumption periods and optimizing load usage.

[4] P. R. Sharma and V. Gupta present an IoT-enabled energy management system for smart industries. The system uses microcontroller-based sensing units to measure electrical parameters such as voltage, current, and power factor. These parameters are transmitted to a cloud-based server where real-time analytics and visualization are performed. The study emphasizes predictive maintenance and energy optimization strategies to improve system performance. The implementation demonstrates improved monitoring accuracy and enhanced control over industrial power usage, contributing to energy conservation and reduced operational costs.

[5] H. Zhang, Y. Liu, and Q. Chen present a remote energy monitoring system using GSM and IoT communication technologies. The proposed system combines GSM-based SMS alerts with internet-based monitoring platforms to ensure reliable communication even in areas with limited internet connectivity. The system automatically sends alerts to users during abnormal power consumption or overload conditions. The integration of GSM technology enhances system reliability and ensures uninterrupted monitoring. The research highlights the importance of hybrid communication technologies in developing robust energy management solutions.

[6] R. Kumar, S. Patel, and A. Verma present a low-cost IoT-based smart electricity monitoring system using Arduino and ESP8266 modules. The system measures real-time power consumption and sends the data to a cloud platform for monitoring and analysis. The study focuses on affordability and ease of implementation, making the system suitable for small industries and commercial establishments. Results demonstrate that the system effectively reduces electricity consumption through better awareness and control of energy usage.

[7] M. Rahman and T. Ahmed present a cloud-integrated smart power monitoring system for industrial applications. The research integrates energy meters with IoT gateways that transmit data to centralized servers for long-term storage and analysis. The system enables operators to monitor multiple industrial machines simultaneously and detect energy inefficiencies. The results show that real-time monitoring and analytics significantly improve operational efficiency and support sustainable industrial practices.

[8] U. Raza, P. Kulkarni, and M. Sooriyabandara present an overview of IoT communication technologies used for energy monitoring applications. The study compares different wireless technologies such as Wi-Fi, GSM, and LPWAN in terms of coverage, energy efficiency, and scalability. The research highlights the importance of selecting appropriate communication technologies to ensure reliable data transmission and efficient system performance in industrial environments.

[9] F. Adelantado, X. Vilajosana, and B. Martinez present a study on scalable IoT architectures for smart metering and energy monitoring systems. The research analyzes system performance under large-scale deployments involving multiple sensors and gateways. The results highlight challenges such as network congestion, latency, and data management. The authors propose optimized network configurations to improve system scalability and reliability in large industrial monitoring systems.

[10] J. Jawad, R. Nordin, and M. Ismail present a comprehensive review of energy-efficient IoT systems for smart industry applications. The study discusses energy monitoring techniques, communication protocols, and data analytics used in modern industrial energy management systems. It emphasizes the role of intelligent automation and IoT integration in reducing energy consumption and supporting sustainable industrial development. The research provides valuable insights for designing cost-effective and scalable smart energy monitoring solutions.

[11] S. Li, L. Da Xu, and S. Zhao present an overview of the Internet of Things (IoT) in industrial applications, focusing on smart energy management systems. The study explores various IoT architectures, sensor technologies, and communication frameworks used for real-time energy monitoring and control. It highlights how IoT-enabled systems improve operational efficiency, reduce energy wastage, and support intelligent decision-making in industries. The research also discusses challenges such as data security, scalability, and interoperability, while providing insights into developing efficient and reliable industrial IoT solutions.

CHAPTER 3

IOT BASED ENERGY MONITORING AND REMOTE LOAD CONTROL FOR SUSTAINABLE INDUSTRY

3.1 Problem Statement

Traditional energy metering systems rely on manual meter reading and provide only cumulative energy consumption data, which makes it difficult for consumers and electricity providers to monitor power usage in real time. These systems lack remote communication, do not support load control, and fail to alert users about excessive energy consumption or abnormal usage conditions.

Due to the absence of real-time monitoring, consumers are often unaware of unnecessary power wastage, leading to increased electricity bills. Additionally, existing meters do not allow users to remotely control electrical loads, which is essential for effective energy management and smart home applications.

Furthermore, most conventional systems do not integrate IoT technologies, limiting data accessibility, analysis, and automation. Systems that rely only on a single communication method, such as GSM or Wi-Fi, face reliability issues due to network or internet unavailability.

Therefore, there is a need for a low-cost, reliable, and intelligent energy monitoring system that can:

- i. Measure energy consumption accurately
- ii. Provide real-time data to users
- iii. Enable remote load control

This project aims to address these challenges by developing an IoT-based smart energy meter using Arduino, GSM, and Wi-Fi technologies, providing efficient energy monitoring and remote control capabilities.

3.2 Existing System

The existing energy metering system mainly consists of conventional electromechanical or basic digital energy meters installed at consumer premises. These meters are used only to measure electrical energy consumption and display the total units consumed. Meter readings are collected manually by electricity board personnel at regular intervals, usually once a month in the traditional system, consumers do not have access to real-time energy consumption data. They can only view the total units displayed on the meter, which makes it difficult to monitor daily or appliance-wise power usage. As a result, energy wastage often goes unnoticed most existing systems do not support remote communication. There is no provision to send consumption data automatically to the user or electricity provider. Any billing, fault detection, or disconnection process requires physical presence, increasing labor cost and time consumption.

Load control is also not available in conventional energy meters. Consumers cannot remotely control electrical appliances or disconnect loads during abnormal conditions such as overconsumption or power theft. In some semi-automatic systems, GSM is used only for sending meter readings via SMS, but these systems lack IoT integration and real-time monitoring. Additionally, traditional meters do not provide alert mechanisms for power overuse, peak load conditions, or abnormal consumption patterns. This limits effective energy management and fails to support smart grid requirements.

3.3 Proposed System

The proposed system introduces an IoT-based smart energy metering solution that overcomes the limitations of the conventional energy metering system by enabling real-time monitoring, remote communication, and load control. The system is designed using Arduino as the main microcontroller, integrated with GSM and Wi-Fi (ESP8266) communication modules.

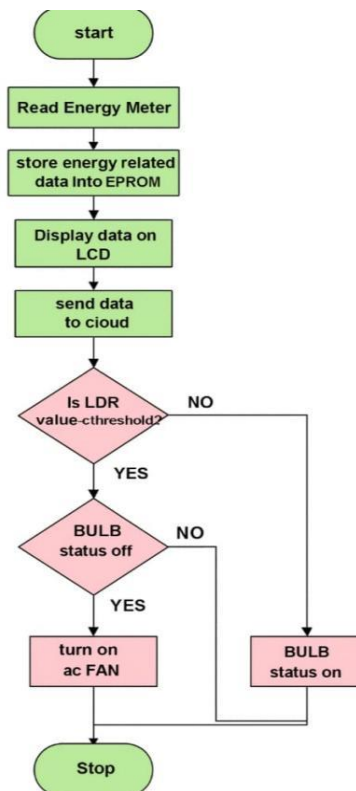


Fig. 3.3. Flow Chart

A single-phase energy meter is used to measure electrical energy consumption. An LDR sensor is placed near the blinking LED of the energy meter to detect light pulses. Each pulse corresponds to a fixed unit of energy, and these pulses are counted by the Arduino to calculate total energy consumption accurately.

The calculated energy data is transmitted using dual communication modes. The ESP8266 Wi-Fi module uploads real-time energy consumption data to a cloud platform, enabling users to monitor usage remotely through a web or mobile application. The GSM module is used to send SMS alerts regarding energy consumption and also to receive user commands. The system provides remote load control functionality using three relay modules connected to three AC loads. Users can switch ON or OFF individual loads by sending predefined SMS commands through GSM communication. This feature helps in reducing power wastage and improving energy management.

CHAPTER 4

HARDWARE COMPONENTS

4.1 Arduino UNO

Arduino UNO is a low-cost, flexible, and easy-to-use programmable open-source microcontroller board that can be integrated into a variety of electronic projects. This board can be interfaced with other Arduino boards, Arduino shields, Raspberry Pi boards and can control relays, LEDs, servos, and motors as an output.

Arduino UNO features AVR microcontroller Atmega328, 6 analogue input pins, and 14 digital I/O pins out of which 6 are used as PWM output the unit comes with 32KB flash memory that is used to store the number of instructions while the SRAM is 2KB and EEPROM is 1KB the operating voltage of the unit is 5V which projects the microcontroller on the board and its associated circuitry operates at 5V while the input voltage ranges between 6V to 20V and the recommended input voltage ranges from 7V to 12V.

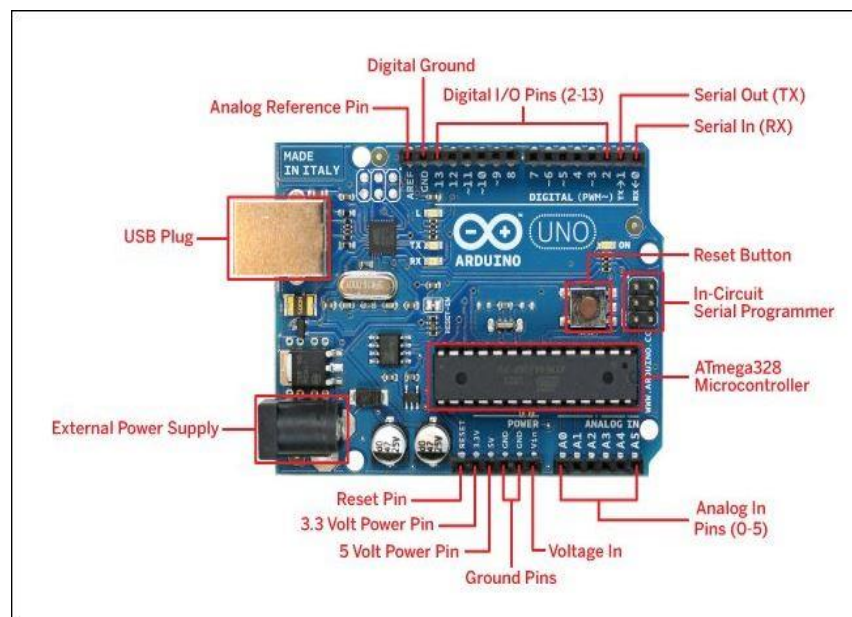


Fig.4.1. Arduino UNO Board

Arduino UNO Specifications

The Arduino UNO board contains the following components and specifications:

- a) **ATmega328**- This is the brain of the board in which the program is stored.
- b) **Ground Pin**- there are several ground pins incorporated on the board.
- c) **PWM**- board contains 6 PWM pins.
- d) **Digital I/O Pins**- 14 digital (0-13) I/O pins available on the board.
- e) **Analogue Pins**- 6 analogue pins integrated on the board.
- f) **AREF**- It is an Analog Reference Pin used to set an external reference voltage.
- g) **Reset Button**- This button will reset the code loaded into the board. This button is useful when the board hangs up, pressing this button will take the entire board into an initial state.
- h) **USB Interface**- This interface is used to connect the board with the computer and to upload the Arduino sketches (Arduino Program is called a Sketch).
- i) **DC Power Jack**- This is used to power up the board with a power supply.
- j) **Power LED**- This is a power LED that lights up when the board is connected with the power source.
- k) **Micro SD Card** -The UNO board supports a micro SD card that allows the board to store more data.
- l) **3.3V**- This pin is used to supply 3.3V power to your projects.
- m) **5V**- This pin is used to supply 5V power to your projects.
- n) **VIN**- It is the input voltage applied to the UNO board.
- o) **Voltage Regulator**- The voltage regulator controls the voltage that goes into the board.
- p) **SPI**- The SPI stands for Serial Peripheral Interface. Four Pins 10(SS), 11(MOSI), 12(MISO), 13(SCK) are used for this communication.

Arduino UNO is easy to program and a person with little or no technical knowledge can get hands-on experience with this board. The Arduino UNO board is programmed using Arduino IDE software which is an official software introduced by Arduino.cc to program the board. The Arduino program is called a sketch which you need to unload into the board. The sketch is nothing but a set of instructions that allow the board to perform certain functions as per your requirements.

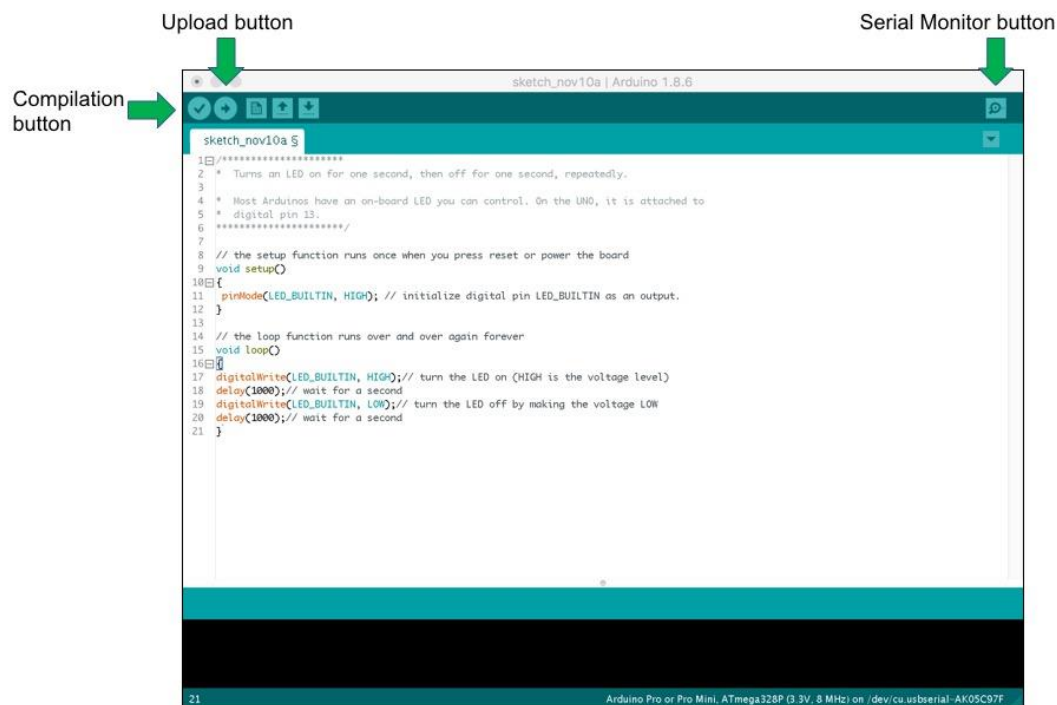


Fig. 4.1. Arduino IDE

Each Arduino sketch comes with two main parts:

1. **void setup()** – this sets up the things that need to be done once and they don't happen again in the running program.
2. **void loop()** – this part comes with the instructions that get repeated again and again until the board is turned off.

Arduino boards can perform some functions that a single microcontroller is capable of doing. But hobbyists and experts still prefer the Arduino board over the microcontroller. Because Arduino boards are easy to use and you don't require a lot of expertise to run these units. Simply plug the board into the computer and start playing with it. While using Arduino boards, you don't require extra peripherals and components to run the boards. Arduino is the complete board that comes with GPIO pins, analogue pins, and a microcontroller as the heart of the board. A microcontroller, on the other hand, is a chip where all the necessary parts like microprocessor, ram, and flash memory are incorporated into a single chip.

So we can say every Arduino board is a microcontroller but not every microcontroller is an Arduino board.

Plus, Arduino is an open-source hardware and software platform which means it is free to use and anyone can modify the boards as per their requirements. Arduino boards can work as a stand-alone project and can be interfaced with other Arduino boards or Raspberry Pi boards. Arduino UNO board is used in the following applications.

- i. Weighing Machines
- ii. Traffic Light Count Down Timer
- iii. Parking Lot Counter
- iv. Embedded systems
- v. Home Automation
- vi. Industrial Automation
- vii. Medical Instrument
- viii. Emergency Light for Railways

Key Specifications

- a) **Microcontroller-** ATmega328P
- b) **Operating Voltage-** 5V
- c) **Input Voltage (rec.)-** 7–12V
- d) **Digital I/O Pins-** 14 (of which 6 provide PWM output)
- e) **Analog Input Pins-** 6
- f) **Flash Memory-** 32 KB (0.5 KB used by bootloader)
- g) **SRAM** - 2 KB
- h) **EEPROM-** 1 KB
- i) **Clock Speed-** 16 MHz
- j) **USB Interface-** USB-B connector
- k) **Communication Protocols-** UART, I2C, SPI

Working Process of Arduino UNO

1. Power Supply

- a) Can be powered via USB, barrel jack, or VIN pin (7–12V).
- b) Internal regulators convert the voltage to a stable 5V and 3.3V for operation.

2. Bootloader and Firmware

- a) The bootloader (pre-installed) allows you to upload code via USB without external programmers.
- b) When powered on, the bootloader runs first, then your uploaded program starts running from flash memory.

3. Reading Inputs

- a) Digital pins (0–13): Read HIGH/LOW (5V/0V) signals from switches, buttons, IR receivers, etc.
- b) Analog pins (A0–A5): Read varying voltages (0–5V) from sensors like temperature, light, or potentiometers using a 10-bit ADC (0–1023).

4. Processing (Logic Control)

- a) The ATmega328P runs code line-by-line.
- b) It can make decisions using if, while, for loops, etc., based on sensor data or inputs.

5. Generating Outputs

- a) Digital Outputs: Control LEDs, buzzers, relays, etc.
- b) PWM Outputs (Pins 3, 5, 6, 9, 10, 11): Control motor speed or LED brightness.
- c) Serial Outputs: Send data to the Serial Monitor or other devices.

6. Communication with External Devices

Uses protocols like UART, I2C, and SPI to interact with modules (GSM, LCDs, SD cards, RTCs, etc).

The Arduino Uno is widely considered the best beginner-friendly and versatile microcontroller board, especially for students, makers, and even early-stage professionals. Here's why:

1. Simple and Easy to Use

- a) Plug-and-play via USB
- b) Beginner-friendly programming with Arduino IDE
- c) No need for external programmer (built-in bootloader)

2. Well-Documented and Supported

- a) Massive open-source community support
- b) Countless tutorials, examples, and forums available
- c) Tons of libraries available for sensors and modules

3. Reliable for Prototyping

- a) Enough GPIO pins for basic to moderate-level projects
- b) Easy to mount on breadboards or custom PCBs
- c) Stable 5V logic compatible with most electronics.

4.2 GSM Module SIM800C

GSM Module SIM800C is a compact, reliable, and cost-effective GSM/GPRS module developed by SIMCom



Fig.4.2. GSM Module

It enables cellular communication for embedded systems and IoT devices, supporting functions like voice calling, SMS, and internet (GPRS) access via standard AT commands.

It is widely used in projects such as:

- SMS-based alerts
- Industrial automation

4.3 Energy meter

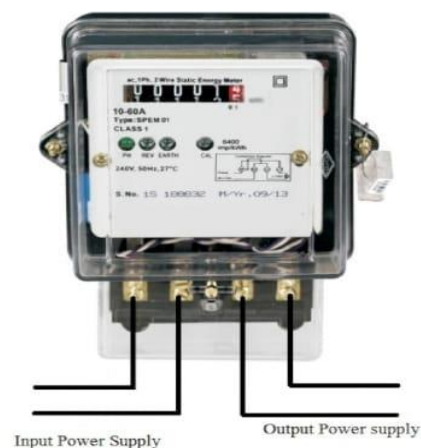


Fig.4.3. Energy Meter

An energy meter is a device used to measure electrical energy (units in kWh) consumed by a load (machine, motor, building, etc.

- 1) Measures how much electricity is used
- 2) Sends data to a microcontroller (ESP8266 / Arduino)
- 3) Data is uploaded to cloud/app
- 4) User can monitor & control loads remotely

In IoT Based Energy Monitoring & Remote Load Control Energy meter is used to

- 1) Measure voltage, current, power, and energy
- 2) Track real-time energy consumption
- 3) Detect over consumption
- 4) Calculate electricity bill
- 5) Enable remote ON/OFF of machines
- 6) Reduce energy wastage
- 7) Improve sustainability

Example

If a machine consumes too much power → system alerts → load can be turned OFF remotely.

- a) Load (machine) consumes power
- b) Energy meter senses:
 - c) Voltage
 - d) Current
- e) Calculates:
 - f) Power (W)
 - g) Energy (kWh)
- h) Sends data to controller
- i) Data shown on:
 - j) LCD
 - k) Mobile app

- l) Web dashboard
- m) Relay controls load remotely

Energy meter plays a vital role in IoT based energy monitoring system by measuring real-time power consumption and enabling remote control of loads. It helps industries reduce energy wastage, lower cost, and achieve sustainability.

4.4 Relay

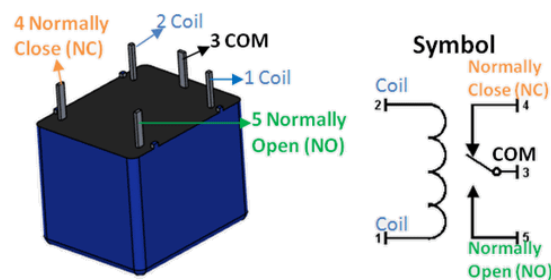


Fig.4.4. Relay

A 12V relay is an electromechanical switch that uses a 12-volt DC signal to control high voltage or high current circuits (like 230V AC or 12V DC load). It allows low-power control (from microcontrollers like Arduino, PIC, etc.) of large power devices like motors, lamps, or appliances.

Components of a Relay

- 1) Electromagnet (Coil): Creates a magnetic field when 12V DC is applied.
- 2) Armature: Movable switch that is pulled by the magnetic field.
- 3) Spring: Returns the armature to its original position when the coil is off.
- 4) Common (COM): Input terminal where the load connects.
- 5) Normally Closed (NC): Connected to COM when the relay is OFF.
- 6) Normally Open (NO): Connected to COM when the relay is ON.
- 7) Contacts: Metal pieces that make or break connection.

Working Principle of 12V Relay

- 1) When no voltage is applied to the coil:
- 2) The armature rests due to the spring.
- 3) COM is connected to NC → Load is OFF (in most cases).
- 4) When 12V DC is applied to the coil:
- 5) The coil becomes electromagnet and pulls the armature.
- 6) The armature shifts from NC to NO.
- 7) COM connects to NO → Load turns ON (light, fan, etc.).
- 8) When power to the coil is removed, the spring returns the armature to NC.

Example Use Cases

You can control a 230V AC bulb using a 5V Arduino and a 12V relay module:

- 1) Arduino sends 5V signal to a transistor.
- 2) Transistor switches 12V to relay coil.
- 3) Relay switches the high voltage circuit ON or OFF.

Relays are most commonly used switching device in electronics. There are two important parameters of relay, first is the Trigger Voltage, this is the voltage required to turn on the relay that is to change the contact from Common → NC to Common → NO. The other parameter is your Load Voltage & Current, this is the amount of voltage or current that the NC, NO or Common terminal of the relay could withstand, in our case for DC it is maximum of 30V and 10A. Make sure the load you are using falls into this range. Above diagram is for relay triggering circuit. Since the relay has 12V trigger voltage we have used a +12V DC supply to one end of the coil and the other end to ground through a switch. For switching we are using a transistor as a switching device. You can also notice a diode connected across the coil of the relay, this diode is called the Fly back Diode. The purpose of the diode is to protect the switch from high voltage spike that can be produced by the relay coil. As shown one end of the load can be connected to the Common pin and the other end is either connected to NO or NC. If connected to NO the load remains disconnected before trigger and if connected to NC the load remains connected before trigger.

4.5 16X2 LCD Display



Fig.4.5. LCD DISPLAY

A 16×2 LCD (Liquid Crystal Display) is a display module that can show:

- 1) 16 characters per line
- 2) 2 lines total

So it can display 32 characters at a time.

It is commonly used in embedded and IoT projects to show:

- 1) Voltage
- 2) Current
- 3) Power
- 4) Energy units
- 5) Status messages

LCD is used to:

- 1) Display real-time energy consumption
- 2) Show voltage and current
- 3) Show power (W) and energy (kWh)
- 4) Display load ON/OFF status
- 5) Show alerts (Overload, Fault, etc.)
- 6) Show system messages

4.6 ESP8266 IT Module



Fig.4.6. ESP8266 Module

The ESP8266 is a low-cost, powerful Wi-Fi module developed by Espressif Systems, widely used in Internet of Things (IoT) applications. It can be used as:

- a) A Wi-Fi modem with AT commands (connected to an external microcontroller like Arduino).
- b) A standalone microcontroller (running user code directly, not covered here).

When used in AT command mode, the ESP8266 communicates over UART (TX/RX) and allows a microcontroller to connect to Wi-Fi networks, perform HTTP requests, and send/receive data wirelessly.

Working of Wi-Fi Module ESP8266

The ESP8266 is a low-cost Wi-Fi-enabled microchip used to connect embedded devices to the internet wirelessly. It can either:

- a) Act as a Wi-Fi adapter for another microcontroller (via AT commands), or
- b) Be used as a standalone microcontroller (running its own code using Arduino IDE, MicroPython, etc.)

ESP8266 Working

- a) Powered with 3.3V
- b) Boots up and loads firmware
- c) Sets mode (STA/AP/STA+AP)

- d) Connects to Wi-Fi using SSID & password
- e) Gets IP address from router
- f) Sends or receives data via internet
- g) (Optional) Runs its own program in standalone mode

Real-World Use Cases

- 1) Control lights from mobile phone
- 2) Upload temperature data to cloud
- 3) Home automation via Blynk or MQTT
- 4) Smart door lock with remote access.

4.7 AC Loads

➤ AC FAN

An AC fan (Alternating Current fan) is a type of electric fan that operates using alternating current (AC) power. It is widely used for ventilation, cooling, and air circulation in household appliances, industrial equipment, and electronic devices. The fan typically consists of a fan blade, stator, rotor, and housing, all powered by an AC motor (usually a single-phase induction motor or shaded pole motor).



Fig.4.7. AC Fan

➤ AC LIGHT

AC Light and AC Bulb Holder



Fig.4.8. Bulb & Bulb Holder

An AC light or AC bulb is an electrical lighting device that operates on alternating current (AC) supply, commonly 110V or 230V, 50/60Hz. It can be an incandescent bulb, CFL (Compact Fluorescent Lamp), or LED bulb.

2. AC Bulb Holder :

An AC bulb holder is an electrical component that holds and connects an AC bulb to the power source. It provides mechanical support and electrical connection.

Common Types:

- a) B22 (Bayonet Cap) – Widely used in India.
- b) E27 (Edison Screw Cap) – Common in Europe and the USA.
- c) Pin-type and Screw-type holders.

➤ 3-PIN AC SOCKET

A **3-pin socket** is an electrical outlet designed to connect **appliances and devices** to the **AC power supply** safely and reliably.



Fig.4.9. AC Socket

CHAPTER 5

HARDWARE IMPLEMENTATION

5.1 Circuit Diagram

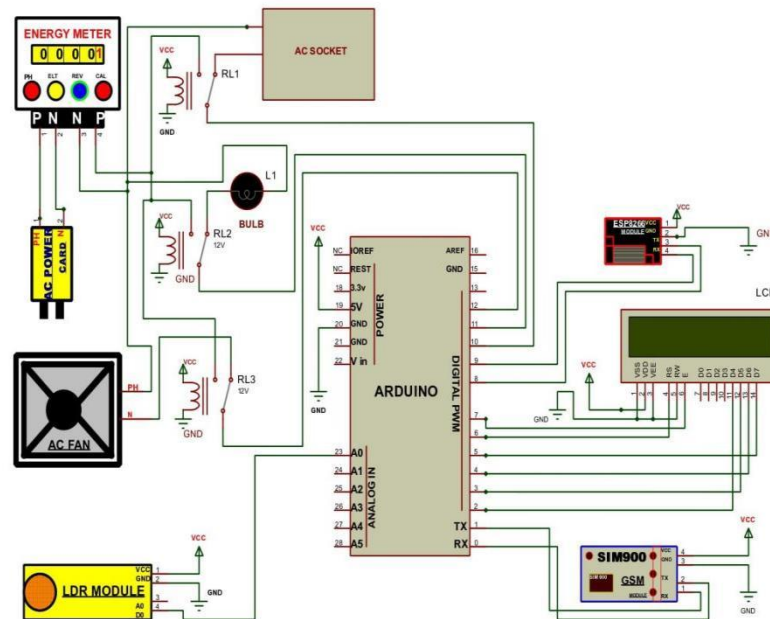


Fig.5.1. Circuit Diagram

1. Power Supply feeds a 1-phase input socket that powers the Load (the industrial equipment).
2. A Relay is placed between the load and an Energy Meter. The relay can switch the load on/off under control.
3. The Arduino UNO is the central controller. It:
 - a) Reads energy data from the Energy Meter.
 - b) Controls the Relay to manage the load.
 - c) Displays information on a 16×2 LCD for local monitoring.
 - d) Communicates with an IoT Module (ESP8266) for remote data transfer and control.

- e) Interfaces with a GSM module (SIM800C) for cellular communication (SMS/alerts or fallback connectivity).

5.2 Hardware Design:

1. Energy Meter: Measures AC power consumption and displays it (PH, ELT, REV, CAL indicators).

2. Arduino: Acts as the central processing unit, handling analog inputs (A0–A5), digital PWM outputs, and communication with peripherals.

3. Load Control: Three relays (RL1, RL2, RL3) manage an AC socket, a bulb (L1), and an AC fan, enabling remote ON/OFF control.

4. Sensors: An LDR module feeds analog data to Arduino pin A0 for light-intensity monitoring.

5. Communication Modules:

A. ESP8266: Provides Wi-Fi connectivity for IoT data transmission.

B. GSM Module (SIM900): Enables cellular communication for remote alerts or control.

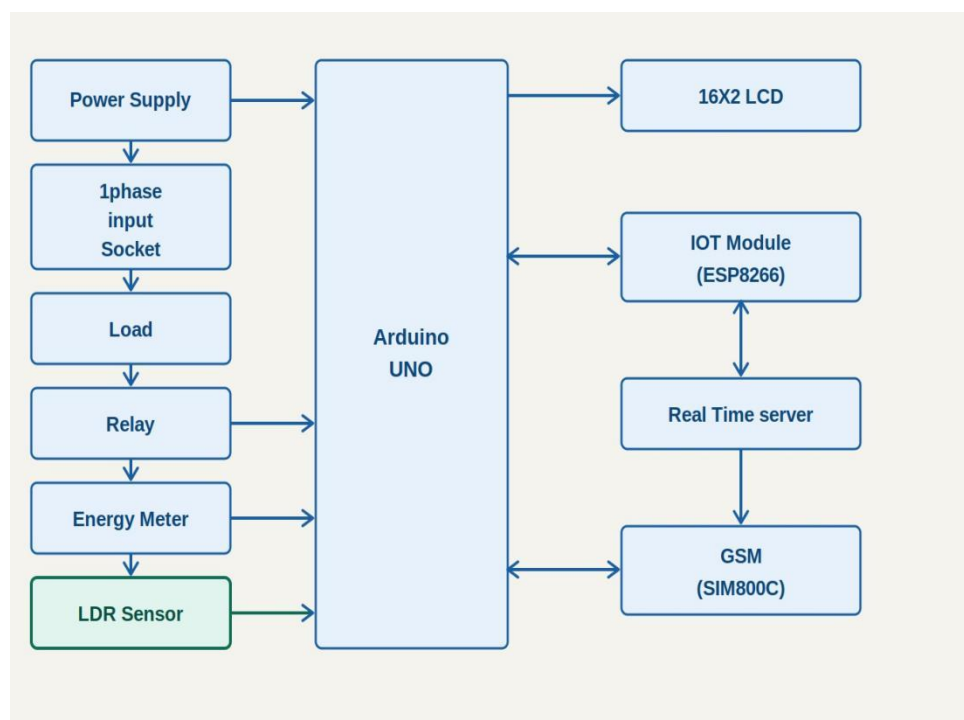


Fig.5.2. Hardware Design

5.3 Output Voltages :

Output voltage plays a crucial role in the proper functioning of the IoT Based Energy Monitoring and Remote Load Control System. Since the system integrates multiple electronic modules such as Arduino UNO, GSM module, ESP8266 WiFi module, LCD display, energy meter, and relays, different voltage levels are required to ensure stable and safe operation. Arduino UNO operates at a regulated 5V DC supply. Stable 5V output is essential because fluctuations may cause microcontroller malfunction or system reset. The ESP8266 WiFi module operates at 3.3V DC logic level. Since it is sensitive to higher voltages, a proper voltage regulator or level shifting circuit is required to step down the voltage from 5V to 3.3V. The GSM SIM900 module typically operates around 4V DC and requires sufficient current during SMS transmission or calling. relay module controls high-voltage AC loads such as lights, fans, and 3-pin sockets operating at 240V AC single phase supply.

Table 5.3 Output Voltages

S.No	Output Source	Voltage Level	Type
1	Arduino UNO	5V DC	Regulated Output
2	GSM Module Logic	4V DC	Communication Level
3	ESP8266 Module	3.3V DC	WiFi Logic Level
4	AC Load Output	240V AC	Single Phase
5	Relay Output	240V AC	Switched Load Output

CHAPTER 6

SOFTWARE IMPLEMENTATION

6.1 Arduino IDE

The Arduino Integrated Development Environment (Arduino IDE) plays a crucial role in the development and implementation of the IoT-based energy monitoring and remote load control system. It is an open-source software platform used for writing, compiling, and uploading code to microcontroller boards such as Arduino Uno, NodeMCU, or ESP8266/ESP32, which form the core of the system. In this project, the Arduino IDE is used to program the microcontroller to read energy parameters such as voltage, current, and power from sensors and transmit the data to a cloud platform via Wi-Fi for real-time monitoring.

The IDE provides a user-friendly interface that includes a code editor, compiler, and serial monitor. The embedded C/C++ programming language is used to write the control algorithms. The program developed in the IDE enables the microcontroller to continuously collect data from energy sensors such as current sensors (e.g., ACS712) and voltage sensors, process the readings, calculate power consumption, and send the information to the IoT server or mobile application. Additionally, the Arduino IDE facilitates debugging through the Serial Monitor, which helps in verifying sensor outputs and system responses during development. The open-source nature of the Arduino IDE allows easy integration of various libraries such as Wi-Fi libraries, sensor libraries, and IoT communication libraries. These libraries simplify communication with cloud platforms and mobile applications. Therefore, the Arduino IDE serves as the backbone for implementing the logic and control mechanisms of the energy monitoring and remote load control system. Arduino UNO is a low-cost, flexible, and easy-to-use programmable open-source microcontroller board that can be integrated into a variety of electronic projects. This board can be interfaced with other Arduino boards, Arduino shields, Raspberry Pi boards and can control relays, LEDs, servos, and motors as an output. Arduino UNO features AVR microcontroller Atmega328, 6 analogue input pins, and 14 digital I/O pins out of which 6 are used as PWM output. The unit comes with 32KB.

6.2 Arduino IDE: Initial Setup

The initial setup of the Arduino IDE is an important step to ensure smooth communication between the computer and the microcontroller used in the project. First, the Arduino IDE software is downloaded from the official Arduino website and installed on the system. After installation, necessary drivers are installed to establish USB communication between the development board and the computer.

For this IoT-based project, additional board support packages are installed depending on the microcontroller used. If NodeMCU (ESP8266) or ESP32 is used for Wi-Fi connectivity, the corresponding board manager URL is added in the preferences section of the Arduino IDE. Then, the required board package is installed through the Board Manager. This enables the IDE to recognize and program the selected IoT board.

Required libraries for energy sensors, Wi-Fi connectivity, and cloud communication are also installed during the initial setup. These libraries include sensor libraries, HTTP or MQTT libraries for IoT communication, and any required API libraries for cloud platforms. Once the libraries are installed, sample test programs are executed to verify the working condition of the board and Wi-Fi connectivity. Proper initial configuration ensures reliable system performance and reduces errors during implementation.

6.3 IDE Board Setup

Board setup in the Arduino IDE ensures that the correct microcontroller model is selected for programming. In this project, the board used may be Arduino Uno for basic energy monitoring or NodeMCU/ESP8266 for integrated Wi-Fi functionality. From the Tools menu in the Arduino IDE, the appropriate board type is selected. For example, if NodeMCU is used, “NodeMCU 1.0 (ESP-12E Module)” is selected under the ESP8266 boards section. Selecting the correct board ensures that the compiler uses the appropriate microcontroller architecture, memory configuration, and clock frequency settings.

6.4 IDE COM Port Setup

The COM port setup establishes communication between the computer and the microcontroller through a USB connection. When the development board is connected to the computer, the operating system assigns a specific COM port number. In the Arduino IDE, this port is selected from the Tools menu under the Port option. Selecting the correct COM port allows the IDE to upload the compiled code to the microcontroller successfully. If an incorrect port is selected, the upload process will fail. The Device Manager in Windows or equivalent system settings can be used to identify the correct port number assigned to the connected board.

The COM port setup also enables serial communication for debugging purposes. Using the Serial Monitor feature in the Arduino IDE, real-time sensor readings such as voltage, current, and power values can be observed. This helps in verifying sensor calibration and ensuring accurate energy measurement. Proper COM port configuration is therefore necessary for programming, testing, and troubleshooting the system.

6.5 Testing

Testing is a critical phase in the software implementation of the IoT-based energy monitoring and remote load control system. After uploading the program to the microcontroller, functional testing is performed to verify each module of the system. Initially, sensor readings are tested individually to ensure accurate measurement of voltage and current. The calculated power and energy values are compared with standard measuring instruments to validate accuracy.

Wi-Fi connectivity is then tested to ensure successful data transmission to the cloud server or mobile application. Real-time data monitoring is verified through dashboards or IoT platforms. Latency, data refresh rate, and connectivity stability are evaluated during this stage. Remote load control functionality is also tested by sending commands from the mobile application or web interface to control connected loads via relay modules.

6. 6 Real Time Server Setup

.A **real-time server** is the backbone of an IoT-based energy monitoring and remote load control system for sustainable industries. It acts as a centralized platform that continuously receives live data from smart energy meters and sensors deployed across industrial equipment. Using protocols like MQTT or HTTP, the server processes parameters such as voltage, current, power consumption, and energy usage in real time. Platforms such as AWS IoT Core, ThingsBoard, or Blynk can be used to handle device communication, data storage, and visualization dashboards. The server also enables intelligent decision-making by applying analytics or threshold-based logic to detect anomalies, peak loads, or energy wastage. Based on these insights, it can remotely control loads—such as turning machines ON/OFF or adjusting operations—to optimize energy efficiency and reduce operational costs. This real-time interaction ensures better energy management, supports sustainability goals, and enhances industrial automation with minimal human intervention.

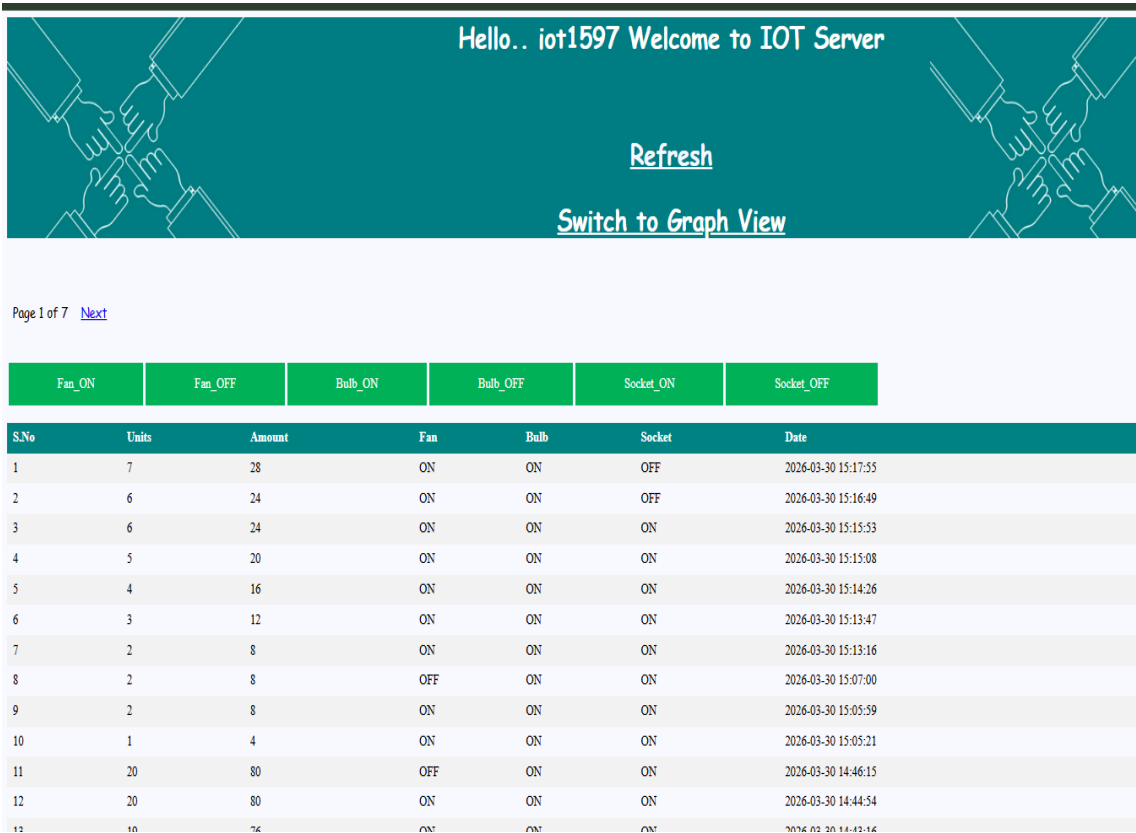


Fig.6.6. Load Monitoring in Server

CHAPTER 7

RESULTS AND DISCUSSION

The result section represents the practical output of the proposed system. After implementing the hardware and software, the system was tested using different electrical loads such as bulbs, fans, and motors. The sensor continuously measured voltage and current, and the controller calculated real-time power and energy consumption.

The measured data was displayed on the 16×2 LCD and also transmitted to the Real Timew Server using Wi-Fi. A web or mobile interface allowed users to observe energy usage in real time and analyze historical data. The relay module enabled remote switching of loads, which proved useful for controlling machines from a distant location.

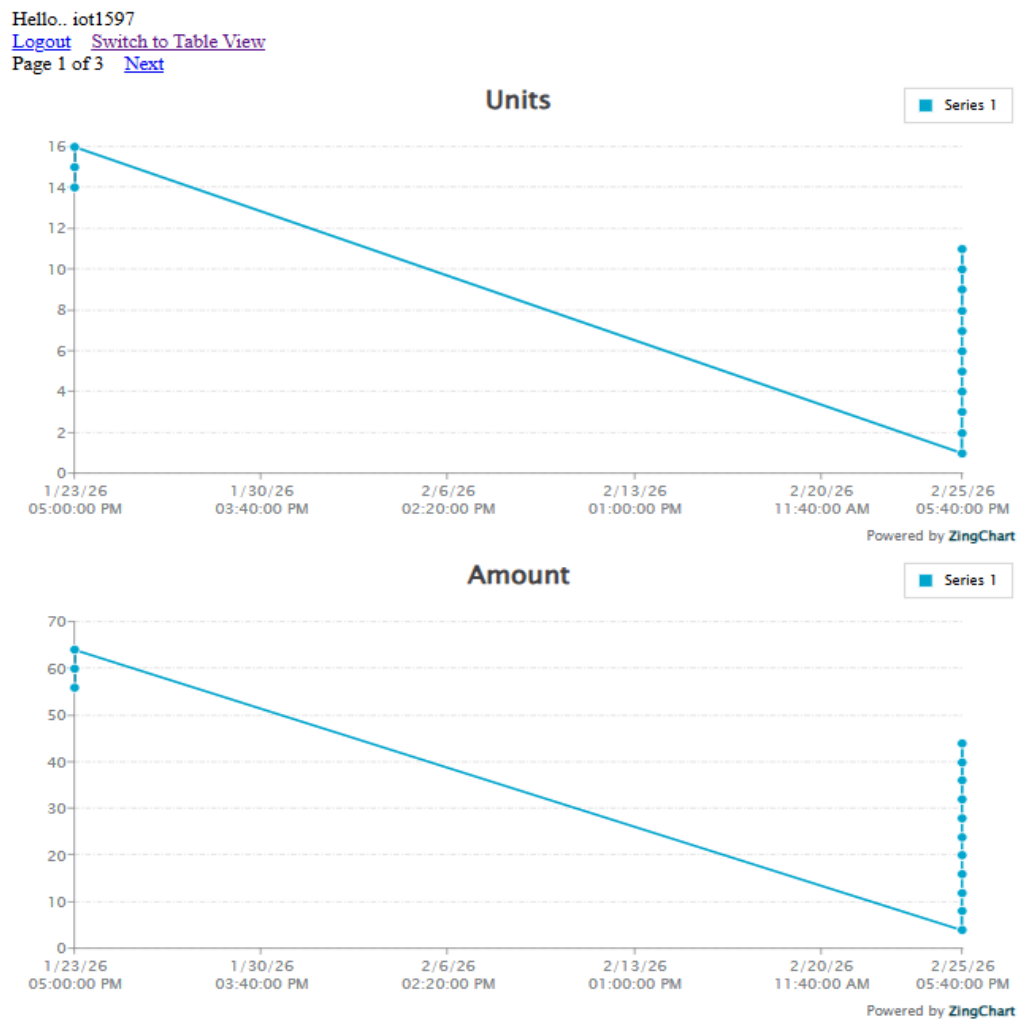


Fig.7.1. System Output

Conclusion

The conclusion summarizes the performance and usefulness of the proposed system. The developed IoT-based energy monitoring and remote load control system effectively measures and manages electrical energy consumption in industrial environments. By providing real-time information and enabling remote control, the system enhances operational efficiency and reduces unnecessary power usage.

The integration of IoT technology allows data to be accessed from anywhere, which supports smart energy management. The system also improves safety by reducing human involvement in high-voltage environments. Overall, the project proves to be a practical solution for achieving sustainable industrial energy utilization.

Future Scope

The proposed system can be extended in several ways in the future:

- a) Addition of artificial intelligence to predict energy demand
- b) Integration with smart grid for automated load control
- c) Implementation of mobile applications for better user interface

Disadvantages

- a) Dependency on Internet Connectivity
- b) Cyber security Risks & Latency

Advantages

- a) Reduction in electricity wastage
- b) Lower operational and maintenance cost

Applications

The proposed system can be applied in:

- a) Industrial plants for machine-level energy monitoring.
- b) Residential buildings for smart metering.
- c) Data centers for monitoring server power usage.

BIBLIOGRAPHY

- [1] A. A. Mirani, A. Awasthi, N. O'Mahony, and J. Walsh, "Industrial IoT-Based Energy Monitoring System: Using Data Processing at Edge," *IoT*, vol. 5, no. 4, pp. 608–633, 2024.
- [2] S. Sulistiyanto *et al.*, "IoT-based model for real-time monitoring of new and renewable energy systems," *EUREKA: Physics and Engineering*, no. 3, pp. 36–48, 2025.
- [3] M. S. Anjana, R. D. Aryadevi, and M. V. Ramesh, "Empowering Sustainability: The Crucial Role of IoT-Enabled Distributed Learning Systems in Reducing Carbon Footprints," *IEEE Access*, vol. 13, pp. 412–428, 2025.
- [4] A. S. Usman, N. Abubakar, and U. M. Jarere, "A low cost IoT based smart energy monitoring and load management system for industrial applications," *FUDMA Journal of Sciences*, vol. 9, no. 1, pp. 122–130, 2025.
- [5] M. Thulasingham and M. Krishnamoorthy, "IoT based intelligent energy monitoring of grid connected hybrid systems and controlling of loads using PLC," *Transactions on Energy Systems and Engineering Applications*, vol. 5, no. 2, pp. 45–59, 2024.
- [6] R. J. S. Raj and S. J. Suji Prasad, "IoT-Based Industrial Energy Management System for Sustainable Manufacturing," in *Proc. Int. Conf. Smart Electronics and Communication (ICOSEC)*, 2023, pp. 412–417.
- [7] O. Munoz, A. Ruelas, and F. Lara, "Design and Development of an IoT Smart Meter with Load Control for Home Energy Management Systems," *Sensors*, vol. 22, no. 19, p. 7536, 2022.
- [8] S. M. A. A. Abir, A. Anwar, and A. S. M. Kayes, "IoT-Enabled Smart Energy Grid: Applications and Challenges," *IEEE Access*, vol. 9, pp. 50961–50981, 2021.
- [9] L. F. S. Silva *et al.*, "An IoT-Based Low-Cost Smart Energy Meter for Industrial Monitoring," in *Proc. IEEE Int. Instrumentation and Measurement Technology Conf. (I2MTC)*, 2020, pp. 1–6.
- [10] N. K. Soni and J. S. Bhadoria, "IoT based energy monitoring and control system," in *Proc. 7th Int. Conf. Computing for Sustainable Global Development (INDIACom)*, 2020, pp. 1–5.
- [11] S. R. Krishnan *et al.*, "Smart Energy Management System using IoT and Cloud Computing for Industrial Sustainability," *Sustainable Computing: Informatics and Systems*, vol. 26, 2020.

PROJECT COs– POs/PSOs MAPPING

Course Title: B.Tech Capstone Project

Course Code: IN3535

Department: Internet of Things

Batch Number: B8

Title of the Project: IoT Based Energy Monitoring and Remote Load Control for Sustainable Industry

Academic Year: 2025 - 2026

1. Project Course Outcomes (COs)

CO No	Course Outcome Statement
CO1	Apply knowledge to design and implement real time solutions for practical Engineering Problems.
CO2	Critically review literature to identify gaps and suggest improvements.
CO3	Conduct research by designing experiments, analyzing data and validating IoT based solutions.
CO4	Design and develop an IoT product considering performance, cost and societal impact.
CO5	Apply ethical principles and communication skills to complete engineering project collaboratively.

2. Program Outcomes (POs)

PO No	Program Outcome Statement
PO1	Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.
PO2	Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)

PO3	Design / Development of Solutions: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)
PO4	Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8).
PO5	Engineering tool usage: Create, select and apply appropriate techniques, resources and modern engineering & 17 tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)
PO6	The Engineer andThe World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7).
PO7	Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)
PO8	Individual & Collaborative Team Work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.
PO9	Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences.
PO10	Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.
PO11	Life Long Learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change. (WK8)

3. Program Specific Outcomes (PSOs)

PSO No	PSO Statement
PSO1	Implementing innovative, costeffective processes for producing energy efficient and eco-friendly IoT applications.
PSO2	Applying the knowledge of Core, Disciplinary and Interdisciplinary elective courses of Electronics, Communications and Computer Engineering to achieve a logical solution to real world problems

4. CO – PO / PSO Mapping Table (3-High, 2- Moderate, 1- Low)

CO/PO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO2
CO1	3	2	1	-	-	-	-	-	-	-	-	2	1
CO2	2	2	3	2	3	-	-	-	1	-	-	3	2
CO3	2	2	2	3	3	-	-	-	1	-	-	3	3
CO4	2	3	2	3	2	-	-	-	1	-	-	2	3
CO5	-	-	1	1	2	-	-	2	3	3	3	1	2

5. Sustainable Development Goals (SDG) Alignment

SDG No	Goal	Relevance to the Project
7	Affordable and Clean Energy	Optimized and automated switching reduces unnecessary energy consumption and supports efficient energy usage in railway operations.
9	Industry, Innovation and Infrastructure	The project enhances railway infrastructure through IoT-based automation, improving efficiency, reliability, and modernization of transport systems.
12	Responsible Consumption and Production:	By providing real-time energy data and remote load control, the project ensures responsible energy consumption and minimizes unnecessary power usage in industrial operations.

Justification of CO - PO/PSO Mapping (Table 5):

CO	Justification
CO1	Understands IoT fundamentals and identifies real-world problems. Supports PO1 and PO2 through application of basic engineering concepts and analysis. Contributes to PSO1 by building domain knowledge.
CO2	Focuses on design and development of IoT systems using modern tools. Strongly maps to PO3 and PO5, while supporting PO2 and PO4. Enhances practical skills aligned with PSO1 and PSO2.
CO3	Involves data acquisition, processing, and cloud integration in IoT. Relates to PO4 and PO5, with support from PO1 and PO2. Develops implementation skills contributing to PSO1 and PSO2.
CO4	Emphasizes analysis and interpretation of IoT data for decision-making. Supports PO2 and PO4, while contributing to PO3. Strengthens analytical ability aligned with PSO2.
CO5	Highlights teamwork, communication, and project management skills. Strongly maps to PO9, PO10, and PO11, while supporting PO8 and PO5. Encourages collaborative IoT development aligned with PSO1 and PSO2.

Signature(s) of Students:

Signature of Project Supervisor:

- 1.
- 2.
- 3.
- 4.

APPENDIX A

#SOURCE CODE

```
#include <LiquidCrystal.h>
#include <stdio.h>
LiquidCrystal lcd(6, 7, 5, 4, 3, 2);
#include <SoftwareSerial.h>
SoftwareSerial mySerial(8,9);
unsigned char rcv;
char pastnumber[11];
int units=0,amount=0;
int sts1=0,sts2=0,sts3=0,sts4=0,sts5=0,sts6=0;
int itk=0;
char gsm_msg[10];
char res[130];
void serialFlush()
{
while(Serial.available() > 0)
{
char t = Serial.read();
}
}
void myserialFlush()
{
while(mySerial.available() > 0)
{
char t = mySerial.read();
}
}
char check(char* ex,int timeout)
{
int i=0;
int j = 0,k=0;
while (1)
{
sl:
if(mySerial.available() > 0)
{
res[i] = mySerial.read();
if(res[i] == 0x0a || res[i]=='>' || i == 100)
{
i++;
res[i] = 0;break;
}
i++;
}
j++;
if(j == 30000)
{
```



```
k++;  
// Serial.println("kk");  
j = 0;  
}  
if(k > timeout)  
{  
//Serial.println("timeout");  
return 1;  
}  
} //while 1  
if(!strcmp(ex,res,strlen(ex)))  
{  
// Serial.println("ok..");  
return 0;  
}  
else  
{  
// Serial.print("Wrong ");  
// Serial.println(res);  
i=0;  
goto sl;  
}  
}  
char buff[200],k=0;  
void upload(unsigned int s1,unsigned int s2,unsigned int s3);  
char readserver(void);  
void clearserver(void);  
const char* ssid = "iotserver";  
const char* password = "iotserver123";  
int sti=0;  
String inputString = ""; // a string to hold incoming data  
boolean stringComplete = false; // whether the string is complete  
int mtr = A0;int button1 = A1;  
int fan = 12;  
int bulb = 11;  
int socket = 10;  
char fan_string[10]="OFF";  
char bulb_string[10]="OFF";  
char socket_string[10]="OFF";  
void okcheck()  
{  
unsigned char rcr;  
do{  
rcr = Serial.read();  
}while(rcr != 'K');  
}  
void setup()  
{  
char ret;  
Serial.begin(1200); //serialEvent();  
mySerial.begin(9600);  
pinMode(mtr, INPUT);
```

```
pinMode(fan, OUTPUT);pinMode(bulb, OUTPUT);pinMode(socket, OUTPUT);
pinMode(button1, INPUT_PULLUP);
digitalWrite(fan, LOW);digitalWrite(bulb, LOW);digitalWrite(socket, LOW);
//IOT smart Energymeter with GSM
lcd.begin(16, 2);lcd.setCursor(0,0);
lcd.print("IOT Smart Energy");
lcd.setCursor(0,1);
lcd.print(" Meter with GSM");
delay(2500);
wifiinit();
delay(2500);
if(digitalRead(button1) == LOW) //GSM
{
  lcd.clear();lcd.print("GSM Initialising");
  gsminit();
}
digitalWrite(fan, HIGH);delay(1000);
digitalWrite(bulb, HIGH);delay(1000);digitalWrite(socket, HIGH);delay(1000);
lcd.clear();
lcd.setCursor(0,0);
lcd.print("U:"); //2-3-4,0
lcd.setCursor(5,0);
lcd.print("A:"); //7-8-9,0
lcd.setCursor(0,1);
lcd.print("F:"); //2-3-4,1 lcd.setCursor(2,1);
lcd.print("ON ");
lcd.setCursor(5,1);
lcd.print("B:"); //7-8-9,1
lcd.print("ON ");
lcd.setCursor(10,1);
lcd.print("S:"); //12-13-14,1
lcd.print("ON ");
memset(fan_string,'\0',strlen(fan_string));
strcpy(fan_string,"ON");
memset(bulb_string,'\0',strlen(bulb_string));
strcpy(bulb_string,"ON");
memset(socket_string,'\0',strlen(socket_string));
strcpy(socket_string,"ON");
itk=0;
//serialEvent();
}
char bf3[50];
int g=0,f=0,count=0,lc=0;
int cntmr=0;
void loop()
{
  if(digitalRead(mtr) == LOW)
  {delay(200);
  while(digitalRead(mtr) == LOW);
  delay(500);
  units++;
  amount = (units * 4);
```

```
lcd.setCursor(2,0); convertl(units);
lcd.setCursor(7,0); convertl(amount);
upload(units,amount,fan_string,bulb_string,socket_string);
if(digitalRead(button1) == LOW) //GSM
{
  lcd.setCursor(15,0);lcd.print("U");
  delay(4000);delay(4000);delay(4000);delay(4000);
  Serial.write("AT+CMGS=\"");
  Serial.write(pastnumber);
  Serial.write("\r\n"); delay(3000);
  Serial.write("U:");
  Serial.print(units);
  Serial.write(" A:");
  Serial.print(amount);
  Serial.write(0x1A); delay(4000);delay(4000);delay(4000);delay(4000);
  lcd.setCursor(15,0);lcd.print(" ");
}
}
while(Serial.available())
{
  char inChar = (char)Serial.read();
  if(inChar == '*')
  {
    sti=1;
    itk=0;
    memset(gsm_msg,'\0',strlen(gsm_msg));
  }
  if(sti == 1)
  {
    gsm_msg[itk] = inChar;
    itk++;
  }
  if(inChar == '#')
  {
    sti=0;
    itk=0;
    stringComplete = true;
  }
}
if(stringComplete)
{
  lcd.setCursor(11,0);lcd.print(gsm_msg);lcd.print(" ");
  if(gsm_msg[1] == 's' || gsm_msg[1] == 'S')
  {
    lcd.setCursor(15,0);lcd.print("R");
    delay(4000);delay(4000);delay(4000);delay(4000);
    Serial.write("AT+CMGS=\"");
    Serial.write(pastnumber);
    Serial.write("\r\n"); delay(3000);
    Serial.write("U:");
    Serial.print(units);
    Serial.write(" A:");
```

```
Serial.print(amount);
Serial.write(0x1A);
delay(4000);delay(4000); delay(4000);delay(4000);
lcd.setCursor(15,0);lcd.print(" ");
}
if(gsm_msg[1] == '1')
{
digitalWrite(fan, HIGH);
lcd.setCursor(2,1);lcd.print("ON ");
memset(fan_string,'\0',strlen(fan_string));
strcpy(fan_string,"ON");
upload(units,amount,fan_string,bulb_string,socket_string);
}
if(gsm_msg[1] == '2')
{
digitalWrite(fan, LOW);
lcd.setCursor(2,1);lcd.print("OFF");
memset(fan_string,'\0',strlen(fan_string));
strcpy(fan_string,"OFF");
upload(units,amount,fan_string,bulb_string,socket_string);
}
if(gsm_msg[1] == '3')
{
digitalWrite(bulb, HIGH);
lcd.setCursor(7,1);lcd.print("ON ");
memset(bulb_string,'\0',strlen(bulb_string));
strcpy(bulb_string,"ON");
upload(units,amount,fan_string,bulb_string,socket_string);
}
if(gsm_msg[1] == '4')
{
digitalWrite(bulb, LOW);
lcd.setCursor(7,1);lcd.print("OFF");
memset(bulb_string,'\0',strlen(bulb_string));
strcpy(bulb_string,"OFF");
upload(units,amount,fan_string,bulb_string,socket_string);
}
if(gsm_msg[1] == '5')
{
digitalWrite(socket, HIGH);
lcd.setCursor(12,1);lcd.print("ON ");
memset(socket_string,'\0',strlen(socket_string));
strcpy(socket_string,"ON");
upload(units,amount,fan_string,bulb_string,socket_string);
}
if(gsm_msg[1] == '6')
{
digitalWrite(socket, LOW);
lcd.setCursor(12,1);lcd.print("OFF");
memset(socket_string,'\0',strlen(socket_string));
strcpy(socket_string,"OFF");
upload(units,amount,fan_string,bulb_string,socket_string);
}
```

```

}
memset(gsm_msg,'\0',strlen(gsm_msg));
inputString = "";
stringComplete = false;
}
delay(100);
/*
if(digitalRead(button1) == HIGH) //WIFI
{
cntmr++;
}
*/
cntmr++;
if(cntmr >= 1000)
{cntmr=0;
lcd.setCursor(15, 1);lcd.print("R");
char ctrl = readserver();
if(ctrl == '1' && sts1 == 0)
{sts1=1;sts2=0,sts3=0,sts4=0,sts5=0,sts6=0;
digitalWrite(fan, HIGH);
lcd.setCursor(2,1);lcd.print("ON ");
memset(fan_string,'\0',strlen(fan_string));
strcpy(fan_string,"ON");
upload(units,amount,fan_string,bulb_string,socket_string);
}
if(ctrl == '2' && sts2 == 0)
{sts1=0;sts2=1,sts3=0,sts4=0,sts5=0,sts6=0;
digitalWrite(fan, LOW);
lcd.setCursor(2,1);lcd.print("OFF");
memset(fan_string,'\0',strlen(fan_string));
strcpy(fan_string,"OFF");
upload(units,amount,fan_string,bulb_string,socket_string);
}
if(ctrl == '3' && sts3 == 0)
{sts1=0;sts2=0,sts3=1,sts4=0,sts5=0,sts6=0;
digitalWrite(bulb, HIGH);
lcd.setCursor(7,1);lcd.print("ON ");
memset(bulb_string,'\0',strlen(bulb_string));
strcpy(bulb_string,"ON");
upload(units,amount,fan_string,bulb_string,socket_string);
}
if(ctrl == '4' && sts4 == 0)
{sts1=0;sts2=0,sts3=0,sts4=1,sts5=0,sts6=0;
digitalWrite(bulb, LOW);
lcd.setCursor(7,1);lcd.print("OFF");
memset(bulb_string,'\0',strlen(bulb_string));
strcpy(bulb_string,"OFF");
upload(units,amount,fan_string,bulb_string,socket_string);
}
if(ctrl == '5' && sts5 == 0)
{sts1=0;sts2=0,sts3=0,sts4=0,sts5=1,sts6=0;
digitalWrite(socket, HIGH);

```

```
lcd.setCursor(12,1);lcd.print("ON ");
memset(socket_string,'\0',strlen(socket_string));
strcpy(socket_string,"ON");
upload(units,amount,fan_string,bulb_string,socket_string);
}
if(ctrl == '6' && sts6 == 0)
{sts1=0;sts2=0,sts3=0,sts4=0,sts5=0,sts6=1;
digitalWrite(socket, LOW);
lcd.setCursor(12,1);lcd.print("OFF");
memset(socket_string,'\0',strlen(socket_string));
strcpy(socket_string,"OFF");
upload(units,amount,fan_string,bulb_string,socket_string);
}
lcd.setCursor(15, 1);lcd.print(" ");
}
}
/*
void serialEvent()
{
while (Serial.available())
{
char inChar = (char)Serial.read();
if(inChar == '*')
{sti=1;
inputString += inChar;
}
if(sti == 1)
{
inputString += inChar;
}
if(inChar == '#')
{sti=0;
stringComplete = true;
}
}
}
*/
int readSerial(char result[])
{
int i = 0;
while (1)
{
while (Serial.available() > 0)
{
char inChar = Serial.read();
if (inChar == '\n')
{
result[i] = '\0';
Serial.flush();
return 0;
}
if (inChar != '\r')
```

```
{
result[i] = inChar;
i++;
}
}
}
}
void gsminit()
{
Serial.write("AT\r\n"); okcheck();
Serial.write("ATE0\r\n"); okcheck();
Serial.write("AT+CMGF=1\r\n"); okcheck();
Serial.write("AT+CNMI=1,2,0,0\r\n"); okcheck();
Serial.write("AT+CSMP=17,167,0,0\r\n"); okcheck();
lcd.clear();
lcd.print("SEND MSG STORE");
lcd.setCursor(0,1);
lcd.print("MOBILE NUMBER");
do{
rcv = Serial.read();
}while(rcv != '*');
readSerial(pastnumber);
lcd.clear();
lcd.print(pastnumber);
pastnumber[10]='\0';
Serial.write("AT+CMGS=\"");
Serial.write(pastnumber);
Serial.write("\r\n"); delay(3000);
Serial.write("Mobile no. registered\r\n");
Serial.write(0x1A); delay(4000);
}
char bf2[50];
void upload(unsigned int unt,unsigned int amt,const char *s3,const char *s4,const char *s5)
{
delay(2000);
lcd.setCursor(15, 1);lcd.print("U");
myserialFlush();
mySerial.println("AT+CIPSTART=4,\"TCP\", \"projectsfactoryserver.in\",80");
delay(8000);
memset(buff,0,strlen(buff));
sprintf(buff,"GET
http://projectsfactoryserver.in/storeddata.php?name=iot1597&s1=%u&s2=%u&s3=%s&s4=%
s&s5=%s\r\n\r\n",unt,amt,s3,s4,s5);
myserialFlush();
sprintf(bf2,"AT+CIPSEND=4,%u",strlen(buff));
mySerial.println(bf2);
delay(5000);
myserialFlush();
mySerial.print(buff);
delay(2000);
mySerial.println("AT+CIPCLOSE");
lcd.setCursor(15, 1);lcd.print(" ");
}
```

```

}
char readserver(void)
{
char t;
delay(2000);
lcd.setCursor(15, 1);lcd.print("R");
myserialFlush();
mySerial.println("AT+CIPSTART=4,\"TCP\", \"projectsfactoryserver.in\",80");
//http://projectsfactoryserver.in/last.php?name=amvi001L
delay(8000);
memset(buff,0,strlen(buff));
sprintf(buff,"GET http://projectsfactoryserver.in/last.php?name=iot1597L\r\n\r\n");
myserialFlush();
sprintf(buff2,"AT+CIPSEND=4,%u",strlen(buff));
mySerial.println(buff2);
delay(5000);
myserialFlush();
mySerial.print(buff);
//read status
while(1)
{
while(!mySerial.available());
t = mySerial.read();
// Serial.print(t);
if(t == '*' || t == '#')
{
if(t == '#')return 0;
while(!mySerial.available());
t = mySerial.read();
// Serial.print(t);
delay(1000);
myserialFlush();
return t;
}
}
delay(2000);
mySerial.println("AT+CIPCLOSE");
lcd.setCursor(15, 1);lcd.print(" ");
delay(2000);
return t;
}
void clearserver(void)
{
delay(2000);
lcd.setCursor(15, 1);lcd.print("C");
myserialFlush();
mySerial.println("AT+CIPSTART=4,\"TCP\", \"projectsfactoryserver.in\",80");
//sprintf(buff,"GET
http://projectsfactoryserver.in/storeddata.php?name=iot1&s10=0\r\n\r\n");
delay(8000);
memset(buff,0,strlen(buff));
sprintf(buff,"GET

```



```
http://projectsfactoryserver.in/storedata.php?name=iot1597&s10=0\r\n\r\n");
myserialFlush();
sprintf(bf2,"AT+CIPSEND=4,%u",strlen(buff));
mySerial.println(bf2);
delay(5000);
myserialFlush();
mySerial.print(buff);
delay(2000);
myserialFlush();
mySerial.println("AT+CIPCLOSE");
lcd.setCursor(15, 1);lcd.print(" ");
delay(2000);
}
void wifiinit()
{
char ret;
st:
mySerial.println("ATE0");
ret = check((char*)"OK",50);
mySerial.println("AT");
ret = check((char*)"OK",50);
if(ret != 0)
{
delay(1000);
goto st;
}
lcd.clear();lcd.setCursor(0, 0);lcd.print("CONNECTING");
mySerial.println("AT+CWMODE=1");
ret = check((char*)"OK",50);
cagain:
myserialFlush();
mySerial.print("AT+CWJAP=\"");
mySerial.print(ssid);
mySerial.print("\",\");
mySerial.print(password);
mySerial.println("\");
if(check((char*)"OK",300))goto cagain;
mySerial.println("AT+CIPMUX=1");
delay(1000);
lcd.clear();lcd.setCursor(0, 0);lcd.print("WIFI READY");
}
void converts(unsigned int value)
{
unsigned int a,b,c,d,e,f,g,h;
a=value/10000;
b=value%10000;
c=b/1000;
d=b%1000;
e=d/100;
f=d%100;
g=f/10;
h=f%10;
```

```
a=a|0x30;
c=c|0x30;
e=e|0x30;
g=g|0x30;
h=h|0x30;
Serial.write(a);
Serial.write(c);
Serial.write(e);
Serial.write(g);
Serial.write(h);
}
void convertl(unsigned int value)
{
    unsigned int a,b,c,d,e,f,g,h;
    a=value/10000;
    b=value%10000;
    c=b/1000;
    d=b%1000;
    e=d/100;
    f=d%100;
    g=f/10;
    h=f%10;
    a=a|0x30;
    c=c|0x30;
    e=e|0x30;
    g=g|0x30;
    h=h|0x30;
    // lcd.write(a);
    // lcd.write(c);
    lcd.write(e);
    lcd.write(g);
    lcd.write(h);
}
```

APPENDIX B







IoT Based Energy Monitoring and Remote Load Control For Sustainable Industry

Dr.Ananda Babu T
Associate Professor, Dept of IoT
Seshadri Rao Gudlavalleru
Engineering College, Gudlavalleru,
Andhra Pradesh, India
anand.mits11@gmail.com

Seelam Santhosh
UG Student, Dept of IoT Seshadri
Rao Gudlavalleru Engineering
College, Gudlavalleru,
Andhra Pradesh, India
santhoshseelam36@gmail.com

Yarramsetty Anusha
UG Student, Dept of IoT
Seshadri Rao Gudlavalleru
Engineering College,
Gudlavalleru,
Andhra Pradesh, India
anushamysself17@gmail.com

Mailapalli Naveen
UG Student, Dept of IoT
Seshadri Rao Gudlavalleru
Engineering College, Gudlavalleru,
Andhra Pradesh, India
naveen.mailapalli23@gmail.com

Peddiboyina Bhanu Sai Teja
UG Student, Dept of IoT Seshadri Rao
Gudlavalleru Engineering College, Gudlavalleru,
Andhra Pradesh, India
bhanusaiteja08@gmail.com

Abstract— Due to the greater demand for electricity today, escalating operational costs, and increasing concern about environmental sustainability, modern industrial settings require efficient use of energy. Conventional energy monitoring systems have poor real-time visibility, lack remote access and do not provide intelligent controls leading to energy lost through waste and improper distribution of loads. This paper describes an Internet of Things (IoT)-based energy monitoring and remote load control system that was developed to improve energy efficiency in industrial settings. The system uses embedded sensors connected to a microcontroller to continuously monitor electrical parameters (i.e., voltage, current, power consumption, energy use) and transmits those parameters to an IoT cloud platform for real-time viewing, data logging, and remote access. The system also provides functionality for remote load control enabling authorized users to turn on/off industrial electrical loads using a web page or mobile device. The integration of real-time monitoring and intelligent controls will assist in responsible decision making; therefore, minimizing energy lost through waste and generally supporting environmentally friendly industrial operations. Experimental verification of the proposed solution indicates its reliability, scalability, and effectiveness in improving energy management for industrial settings.

Keywords— IoT, Energy Monitoring, Smart Industry, Remote Load Control, Power Management, Sustainable Industry

1.INTRODUCTION

There is a very basic relationship between energy consumption and growth within the industrial world, as well as between electrical energy resources and the stability of economies and the advancement of technologies. In addition, with industrialisation world wide continuing to accelerate, so too has global consumption of energy resources, which has placed additional stresses on power generation resources and has contributed to increased operational, and ultimately production costs for many industries [2], [4]. As a result, and because of the increased awareness of environmental issues, like carbon emissions and resource depletion, the need for more efficient and sustainable energy use has become one of the highest priorities for economic and environmental sustainability. Many industrial facilities are still reliant on traditional methods of monitoring and managing energy use, such as manual meter readings, and/or by regionalised methods of monitoring systems that provide limited visibility into real-time use of electrical power and do not provide the necessary action-based data for achieving energy management on an ongoing basis. Also, when industrial facilities lack the continuous monitoring and the ability to apply intelligent analytics to their systems, there is little or no opportunity for timely abnormalities to be detected, inefficient loads to be operated and wasted energy to be consumed [1]. Additionally, a lack of remote access often prohibits timely intervention when there are problems, particularly in larger or geographically distributed industrial facilities.

The Internet of Things (IoT), energy management for the industrial sector has undergone a revolutionary change [1], [3]. The IoT creates an infrastructure where sensor-based devices, embedded controllers, communication networks, and cloud-based data storage can communicate, providing a seamless flow of data in real-time and increasing visibility into a given system's operation. IoT-enabled energy monitoring systems allow for continuous measurement and transmission of electrical parameters—including voltage, current, power, and energy consumption—to centralized data storage platforms.

2.RELATED WORK

Recent advancements in IIoT have radically changed the way we monitor and manage energy in industrial settings. Many researchers have created a variety of IoT frameworks to enhance energy efficiency, sustainability, and operational transparency. An example of this was offered by Mirani et al. [1], who created an energy monitoring system based on IIoT that employs edge computing for preprocessing energy data on site before transmitting it to the cloud. This method reduces latency and bandwidth consumption while enabling near real-time analytics for industrial use. They showed that processing in distributed fashion enhances the scalability and responsiveness of intelligent industrial systems.

Additionally, Sulistiyanto et al. [2] created a model for real-time monitoring of renewable energy systems based on IoT. This monitoring system enables continuous tracking of voltage, current, and power parameters by using sensor networks combined with cloud-based platforms. The authors highlighted the importance of fleet management and centralized monitoring in achieving efficient energy use. Another area of exploring energy sustainability involved intelligent distributed learning systems.

Anjana et al. [3] considered how IoT-enabled distributed learning frameworks can reduce carbon footprints in the area of industrial and energy systems. Their research provides evidence that predictive analytics and AI-optimized processes can significantly reduce unnecessary energy use while supporting environmentally-forward operations.

Usman et al. [4] developed an inexpensive IoT based smart energy monitoring and load management system that can be effectively used in small-scale industries. This system uses a wireless communication module with microcontroller based sensors to manage the loads remotely. The results showed that an inexpensive IoT solution could significantly reduce energy consumption without the need for expensive equipment and system infrastructure.

Thulasingham and Krishnamoorthy [5] developed an IoT-based intelligent monitoring system to monitor the production of energy using grid-connected hybrid energy systems and controlling the load on these energy sources using programmable logic controllers (PLCs). Their results showed that using real-time load switching would greatly increase the reliability of the energy produced by these hybrid energy systems and therefore optimize the distribution of that energy throughout the industrial environment.

Raj and Suji Prasad [6] developed an IoT based energy management framework for the purpose of sustainable manufacturing in the manufacturing sector. This system provides real-time dashboard access, automated alerts, and centralized supervisory access to provide users with greater transparency in their energy usage, thereby improving the ability for users to make informed decisions about their energy usage

.Munoz et al. [7] created a smart metering device based on IoT that can regulate load within a home energy management system by controlling how much energy is being consumed from the electrical grid to provide power to an entire home. This research, while focused primarily on residential applications, established an expandable method for implementing load control in an industrial setting.

Additionally, Abir et al. [8] addressed the larger issue of implementing IoT smart energy grids and demand-response strategies for increasing stability in the electrical grid and also for integrating renewable energy sources while using intelligent load balancing.

3.PROPOSED METHODOLOGY

The objective of the proposed system is to develop an IoT solution for energy management with remote load control capabilities for sustainable use in industrial settings. The system's design combines real-time acquisition of electrical parameters, cloud transfer of data, centralized reporting, and remote control of electrical loads into a single architecture.

The entire system is composed of four main components: energy sensing module; processing unit and control; communications unit; and real-time server platform. The energy sense module is responsible for continuously measuring key electrical and energy parameters (voltage, current, active power, and total electrical energy consumed). The energy meter interface provides an extremely accurate way of acquiring electrical consumption data from industrial loads, allowing for an effective method of monitoring operational parameters accurately. These measured values are the basis for intelligent decision-making on energy optimization.

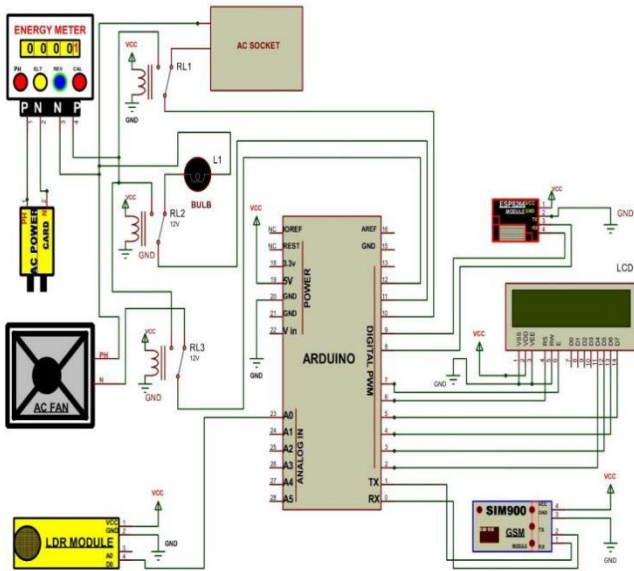


Fig.1.Circuit Diagram

The processing and control unit (which uses an embedded microcontroller) performs real-time acquisition of data, calculations on parameters, evaluation of thresholds, and execution of control logic. In order to accomplish this, it continuously analyses electrical data as it comes in, and compares it to predetermined safety and operational limits. If there is excessive electrical consumption or overload conditions detected, automatic load control actions will be implemented by the system in order to eliminate waste of energy and to protect equipment from damage. This intelligence integrated into the embedded software improves the speed and reliability of the systems.

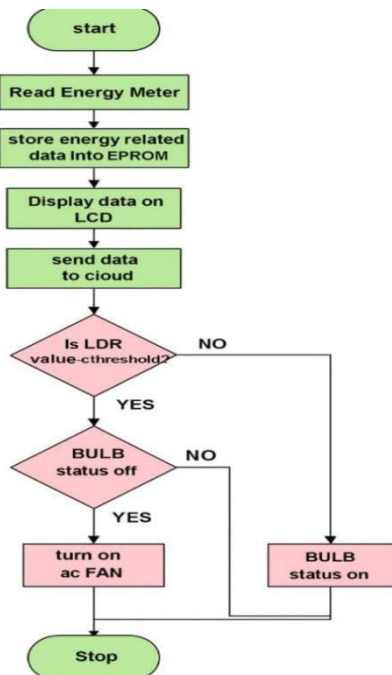


Fig.2.Flow Chart

3.1 Data Acquisition and Energy Measurement:

The Data Collection and Power Measurement module is the first layer of the proposed IoT-based industrial energy monitoring system. The accurate and continuous measuring of electrical parameters is vital to enable the monitoring, intelligent control, and sustainability of energy management.

The Data Acquisition Module is intended to continuously loop to allow for ongoing monitoring of industrial loads. Once measurement of the load has been taken, the Data Acquisition Module packages the measurement into a structured data packet and transmits that data packet to the Communications Module for integration with cloud services. Each data packet has a date/time stamp, which provides accurate information for historical evaluations and for viewing trends on the real-time server system.

3.2 Data Processing and Control Unit:

The Processing and Control Unit offers central intelligence to the IoT-based Energy Monitor and Load Control System. An example of this architecture would be using an Arduino as the primary embedded controller (microcontroller) that will process data in real-time, make decisions, and take control actions based on those decisions. The microcontroller continuously receives electrical parameters or values (voltage, current, calculated power) from the energy meter; it performs calculations such as calculating real-time power, accumulating energy, and comparing with thresholds.

By integrating real-time calculations, intelligent threshold-based control and execution of remote commands, the Processing and Control Unit provides a responsible, responsive and sustainable means of managing industrial energy.

3.3 Remote Load Control Strategy:

The Remote Load Control Strategy is an essential functional component of the IoT-based energy monitoring system to support operations and prevent unnecessary use of energy in an industrial facility. By providing real-time monitoring of power consumption, the proposed system measures energy efficiency, but to achieve this, the correct regulation of electrical loads must also occur in a dynamic fashion. The proposed system provides for manual/automatic remote load control through an embedded computer architecture - connected to the cloud. there are typically many electrical loads used within an industrial facility such as motors, compressors, lighting, heating, and auxiliary equipment.

In many cases, these electrical loads continue to operate and consume energy during off production hours, leading to wasted energy. to address this issue, the system includes relay modules (RL1, RL2, RL3) that are interfaced with the Arduino microcontroller to allow for electrical isolation/switching of the connected electrical loads. Each of these loads is represented as binary control (ON/OFF) variable.

3.4 Communication and Real-Time Server Integration

The communication and real-time server integration module is essential to the proposed IoT-based energy monitoring and load control system by providing the capability for remote monitoring of users, centralized supervision, and bidirectional control of systems. ESP8266 WiFi module enables real-time data transmission to a cloud-based server. The measured parameters (voltage, current, power, energy) are sent to the server using IoT communication protocols such as HTTP or MQTT. In addition, this module allows the transmission of data from embedded hardware in an industrial setting to a cloud server (accessible to users with permission) in a seamless and real-time manner. This will help prevent any unauthorized manipulation of the loads in the industrial sector as well as protect sensitive operation data.

4. REAL TIME SERVER SETUP

In the proposed system, an IoT cloud platform will manage industrial energy consumption data in real time (collecting, storing, visualizing and controlling). In this application, energy data such as Voltage, Current, Power and Total Energy consumption, will be sent to the Real Time Server using ESP8266 Wi-Fi module for wifi connection between Embedded Controller and Cloud Server. After finalizing the IoT cloud server, an Initial IoT Cloud Dashboard will be created and unique authentication credentials (API key / device token) assigned to all users. This information will be embedded into the microcontroller programming code for secure communication between the microcontroller and cloud server.



Switch to Graph View

Page 1 of 3

S.No	Units	Amount	Fan	Bulb	Socket	Date
1	3	12	OFF	OFF	OFF	2025-12-30 11:31:51
2	3	12	OFF	OFF	ON	2025-12-30 11:31:34
3	3	12	OFF	ON	ON	2025-12-30 11:31:18
4	3	12	ON	ON	ON	2025-12-30 11:30:25
5	2	8	ON	ON	ON	2025-12-30 11:30:03
6	2	8	ON	ON	OFF	2025-12-30 11:29:11
7	1	4	ON	ON	OFF	2025-12-30 11:28:12
8	0	0	ON	ON	OFF	2025-12-30 11:27:51
9	0	0	ON	OFF	OFF	2025-12-30 11:26:52
10	2	8	ON	ON	ON	2025-12-30 11:22:30
11	1	4	ON	ON	ON	2025-12-30 11:22:07
12	1	4	ON	ON	ON	2025-12-30 11:21:50
13	1	4	ON	ON	OFF	2025-12-30 11:20:58
14	0	0	ON	OFF	OFF	2025-12-30 11:20:22
15	1	4	ON	ON	OFF	2025-12-29 23:25:35
16	1	4	ON	OFF	OFF	2025-12-29 23:24:37
17	0	0	ON	OFF	OFF	2025-12-29 23:23:47
18	0	0	ON	OFF	ON	2025-12-29 23:22:58
19	0	0	ON	OFF	OFF	2025-12-29 23:22:03
20	1	4	OFF	OFF	OFF	2025-12-29 23:19:17

Fig3.Real Time Server Setup

The following will be completed by the real time server:

1. The Real Time Server will receive real-time energy parameters from the industrial node.
2. The Real Time Server stores time stamped data in a cloud database and provide graphical visualization of Voltage, Current and Power trends.
3. The Real Time Server will generate alerts if usage exceeds predetermined levels and will send remote ON/OFF commands to control industrial loads.

5. HARDWARE IMPLEMENTATION:

The hardware design is created from a schematic diagram that includes all elements required for sensing, processing, communicating, displaying, and controlling; therefore, each individual part serves an independent function. Hardware architecture consists of an integrated processing module, communication modules, control modules and display modules into one complete system.



Fig4.Hardware Implementation

a. **Arduino Microcontroller** : The Arduino microcontroller is the central controlling unit. It has the capability of reading sensor values as well as determining some electrical values (e.g., V, I, P, and E), processing control logic, and communicating via the ESP8266. It will also control relay states for load switching.

b. **Relay Module** : The purpose of using the relay module is to switch industrial loads either to an energized (ON) or de-energized (OFF) state. The relay acts as a simple electrically isolated switch that is controlled by the Arduino with a change in state controlled by a control signal provided by the server and/or when a set of threshold conditions are met.

c. **ESP8266 Wi-Fi Module** : The ESP8266 provides a means for the embedded system to connect to the internet. Therefore, it gives the ability for wireless communication to occur between the hardware elements of the embedded electronics and a cloud server; thus, allowing real-time data transmission to report streaming data to the Cloud Server and remote user command transmission back to the hardware.

d. **LCD Display** : The local LCD is used to display real-time values (e.g, E) for periodic monitoring/review without having to access the internet. The display is used to monitor the data on-site.

e. **Energy Meter** : To measure total energy consumed by the connected loads (one supplied or to be supplied) a single-phase energy meter is employed.

f. **Loads** : The system utilizes a variety of different electrical loads for evaluating how well the system will perform in real-world operating conditions. An AC motor-powered fan, AC bulb Additionally, a three-pin socket is provided as an external load port that allows the user to connect and test a variety of industrial under real-time operating conditions, thereby allowing for greater flexibility and applicability of the system.

g. **GSM Module** : The GSM module functions by sending alert notifications and status updates to the user via SMS, adding reliability to the system due to the module providing communication support regardless of internet connectivity. Therefore, the user can continue to monitor the system and receive notifications of faults.

h. **LDR Module** : The LDR is used to measure the ambient light intensity and is considered a light-sensitive variable resistor; as the resistance of the LDR is determined by the amount of light exposed to it, it can automatically control or measure lighting conditions to improve the efficiency of energy usage.

5. RESULTS

The experimental evaluation of an IoT-based energy monitoring and remote load control system was performed with a hardware prototype. This prototype includes the following components: an Arduino microcontroller, an ESP8266 Wi-Fi module, a single-phase energy meter, a relay module, an LCD display, a GSM module, and three loads (a fan, a bulb, and a three-conductor plug). System performance was checked via real-time data collection, cloud-based visualization, accuracy of load control, and ability to monitor loads remotely.

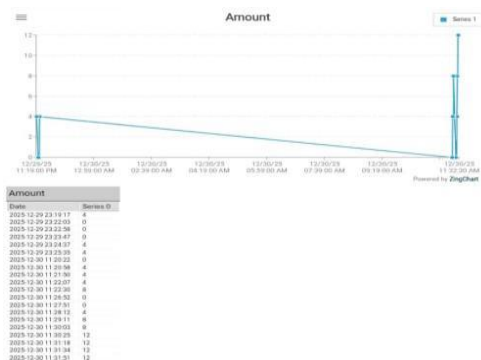


Fig5.Amount for Consumed Units

As shown in the "Amount" graph, the calculated cost of the energy used is represented by the number of units consumed. The periodic increases in price indicate that the microcontroller successfully implemented an energy usage calculation and billing scheme.

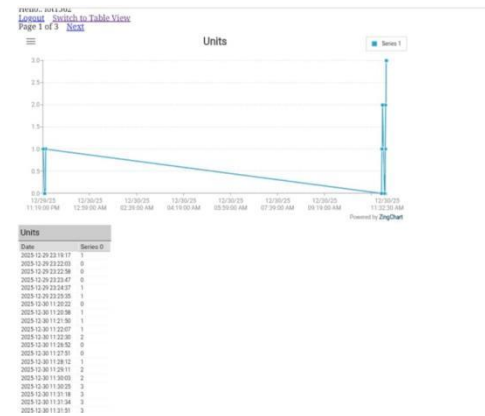


Fig6.No.of Units Consumed

The real-time data was successfully transmitted from the embedded system to the server platform. As shown in the "Units" graph, data collected from the loads being turned on and off accounts for periodic increases in the energy used over time. Corresponding to those time periods when higher-power rated loads (fan, and external 3-conductor plug load) were activated, there were instantaneous increases in usage recorded by the system.

6. CONCLUSION:

The study introduced an energy monitoring and load control system that utilizes an integrated IoT cloud server within a sustainable industrial application. Prior to this device, operators did not have an immediate view into their processes nor the ability to remotely control them, resulting in ineffective management of their resources. The proposed design integrates all aspects of the device into a single system comprised of the following functional capabilities; sensing modules, an Arduino-based processor, ESP8266 Wi-Fi communication, relay-based load controlling units, and a real-time server platform. The experimental results confirmed the real-time capabilities of the device, whereby voltage, current, power, and cumulative consumption were able to be accurately tracked.

ACKNOWLEDGEMENT:

The authors express their sincere gratitude to the Department of Internet of Things, Seshadri Rao Gudlavalluru Engineering College (Autonomous), Gudlavalluru, for providing the necessary facilities and technical support to carry out this work. The authors also thank their project guide, Dr. Ananda Babu T, Associate Professor, Department of Internet of Things, for his valuable guidance, continuous encouragement, and constructive suggestions throughout the development of this project.

REFERENCES

- [1] A. A. Mirani, A. Awasthi, N. O'Mahony, and J. Walsh, "Industrial IoT-Based Energy Monitoring System: Using Data Processing at Edge," *IoT*, vol. 5, no. 4, pp. 608–633, 2024.
- [2] S. Sulistiyanto *et al.*, "IoT-based model for real-time monitoring of new and renewable energy systems," *EUREKA: Physics and Engineering*, no. 3, pp. 36–48, 2025.
- [3] M. S. Anjana, R. D. Aryadevi, and M. V. Ramesh, "Empowering Sustainability: The Crucial Role of IoT-Enabled Distributed Learning Systems in Reducing Carbon Footprints," *IEEE Access*, vol. 13, pp. 412–428, 2025.
- [4] A. S. Usman, N. Abubakar, and U. M. Jarere, "A low cost IoT based smart energy monitoring and load management system for industrial applications," *FUDMA Journal of Sciences*, vol. 9, no. 1, pp. 122–130, 2025.
- [5] M. Thulasingham and M. Krishnamoorthy, "IoT based intelligent energy monitoring of grid connected hybrid systems and controlling of loads using PLC," *Transactions on Energy Systems and Engineering Applications*, vol. 5, no. 2, pp. 45–59, 2024.
- [6] R. J. S. Raj and S. J. Suji Prasad, "IoT-Based Industrial Energy Management System for Sustainable Manufacturing," in *Proc. Int. Conf. Smart Electronics and Communication (ICOSEC)*, 2023, pp. 412–417.
- [7] O. Munoz, A. Ruelas, and F. Lara, "Design and Development of an IoT Smart Meter with Load Control for Home Energy Management Systems," *Sensors*, vol. 22, no. 19, p. 7536, 2022.
- [8] S. M. A. A. Abir, A. Anwar, and A. S. M. Kayes, "IoT-Enabled Smart Energy Grid: Applications and Challenges," *IEEE Access*, vol. 9, pp. 50961–50981, 2021.
- [9] L. F. S. Silva *et al.*, "An IoT-Based Low-Cost Smart Energy Meter for Industrial Monitoring," in *Proc. IEEE Int. Instrumentation and Measurement Technology Conf. (I2MTC)*, 2020, pp. 1–6.
- [10] N. K. Soni and J. S. Bhadoria, "IoT based energy monitoring and control system," in *Proc. 7th Int. Conf. Computing for Sustainable Global Development (INDIACom)*, 2020, pp. 1–5.
- [11] S. R. Krishnan *et al.*, "Smart Energy Management System using IoT and Cloud Computing for Industrial Sustainability," *Sustainable Computing: Informatics and Systems*, vol. 26, 2020.
- [12] G. J. Sowmya and S. Lakshmanan, "Implementation of IoT based smart energy meter for remote monitoring and load control," *International Journal of Engineering and Advanced Technology*, vol. 9, no. 3, 2020.
- [13] H. Al-Badi and A. Al-Lawati, "Design of IoT-Based Smart Energy Meter for Green Buildings," in *Proc. IEEE Int. Conf. Smart Grid and Smart Cities (ICSGSC)*, 2019, pp. 78–82.
- [14] V. Agarwal, A. Gureja, and R. Singh, "IoT Based Smart Energy Metering and Power Theft Control," in *Proc. 4th Int. Conf. Internet of Things: Smart Innovation and Usages (IoT-SIU)*, 2019, pp. 1–5.
- [15] S. S. Shravani and S. K. Sood, "IoT-based framework for energy consumption forecasting and monitoring in sustainable industries," *Sustainable Computing: Informatics and Systems*, vol. 21, pp. 120–132, 2019.